From Nature's Control Grammar to Stable AI Agents : Purpose-Matched Control Subsets, 4π Closure, and Ledgered Self-Improvement

Integrating Twelve Physics-Derived Controls, Proto-Eight Actuation, Hidden-Frame Commitment, and Admissible Runtime Revision

---

Abstract

AI Agent architecture is commonly described through components: planner, memory, retriever, tool interface, evaluator, verifier, and safety layer. These components are useful, but their presence does not establish that an agent is stable. A system can contain every fashionable module and still drift from its purpose, lose evidence context, cross permission boundaries, commit too early, conceal unresolved contradictions, or learn from accidental success.

This article proposes a different starting point. Rather than asking only which components an agent contains, it asks which **control functions** must be present for a bounded agent to remain coherent while acting, committing, recovering, and improving.

The proposed framework combines four complementary structures.

First, twelve controls abstracted from fundamental physics provide a candidate high-level stability repertoire: symmetry, frame invariance, conservation, quantization, exclusion, transition thresholds, locality, least action, topology or holonomy, decoherence, binding, and transition gates.

Second, Proto-Eight Dynamics supplies an actuation grammar. Gradient, gate, boundary, exchange, trigger, guidance, memory, and focus explain how an agent converts latent possibility into directed action, retained structure, and renewed capacity.

Third, 4π Closure supplies a global commitment condition. A visible result may appear complete while its evidence, assumptions, permissions, execution path, or residual uncertainty remain twisted. A 4π-centered agent therefore does not commit merely because the endpoint matches the request. It commits only when the endpoint and hidden execution frame close together.

Fourth, ledgered declaration and admissible self-revision explain how one completed episode changes the next. Trace and residual do not merely document past activity. They revise the future boundary, feature map, gate, evidence rule, and action policy. Improvement is therefore not unrestricted self-modification. It is trace-preserving, residual-honest, frame-robust, budget-bounded revision.

The resulting architecture is:

Purpose Declaration → Protocol Compilation → Control-Subset Selection → Proto-Eight Actuation → Execution → Closure Audit → Trace + Residual → Recovery or Admissible Revision. (0.1)

The article's central thesis is:

A stable AI Agent is a bounded, purpose-bearing runtime that selects an adequate subset from a broad control repertoire, acts through declared operational primitives, commits through an

appropriate closure mode, preserves trace and residual in a ledger, and revises its future declaration without falsifying its past. (0.2)

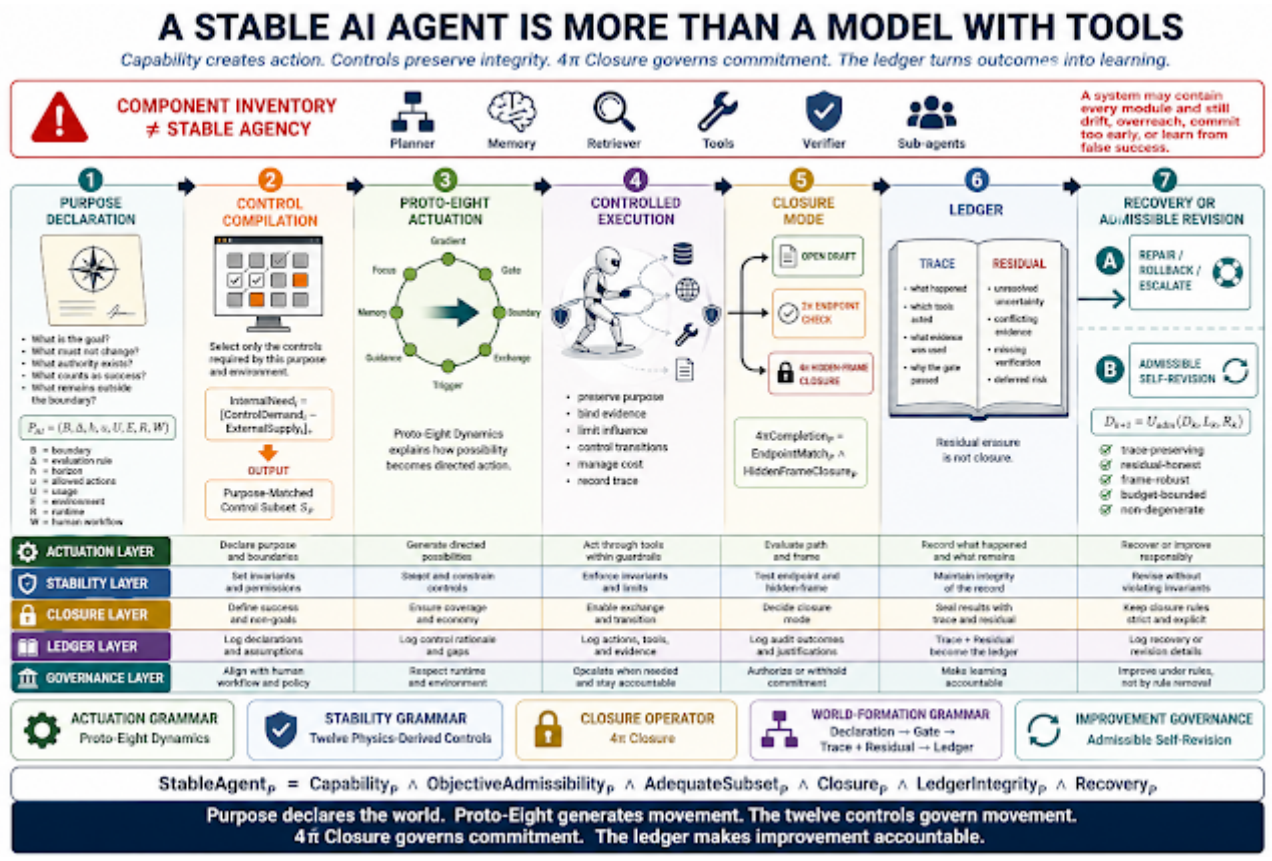
In compact form:

StableAgent_P = Capability_P ∧ AdequateSubset_P ∧ Closure_P ∧ LedgerIntegrity_P ∧ Recovery_P. (0.3)

For an improving agent:

ImprovingAgent_P = StableAgent_P ∧ AdmissibleRevision_P(L_P,R_P). (0.4)

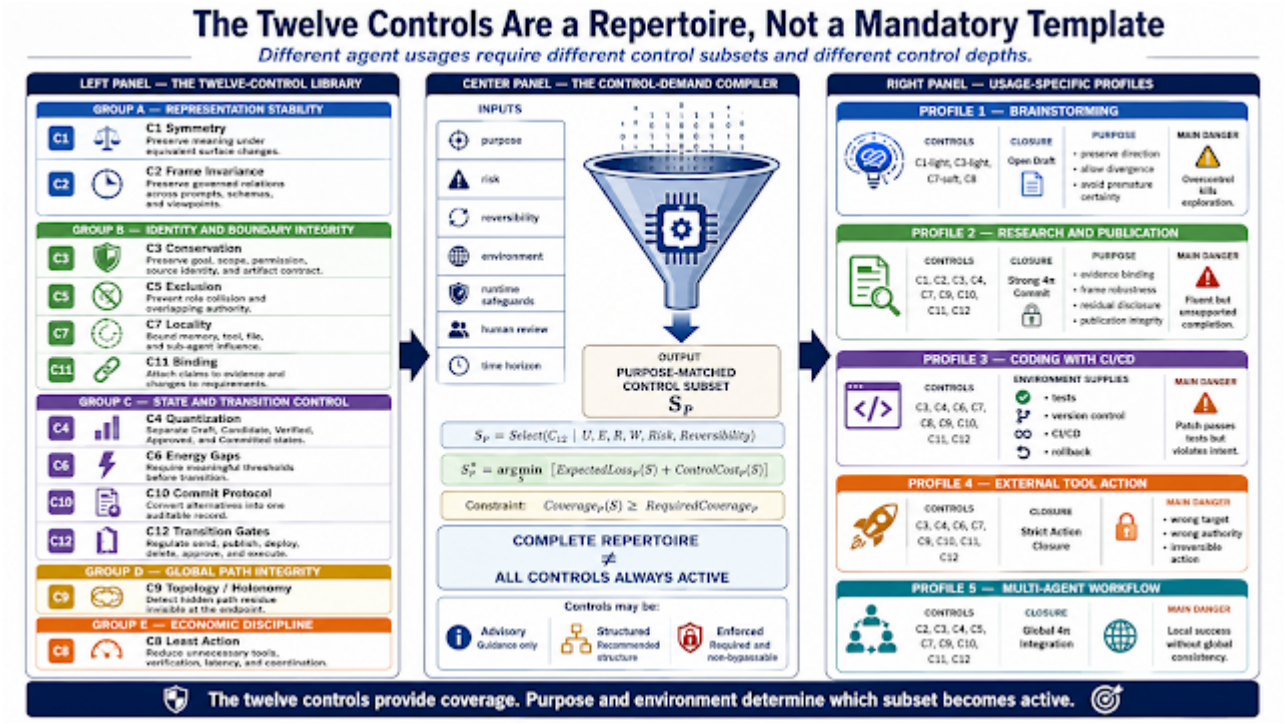
The framework does not claim that AI Agents are literally quantum systems. It claims that nature supplies a mature grammar of persistent identity, bounded interaction, selective transition, binding, trace formation, invariance, and closure. AI engineering may learn from that grammar without confusing analogy with substance.

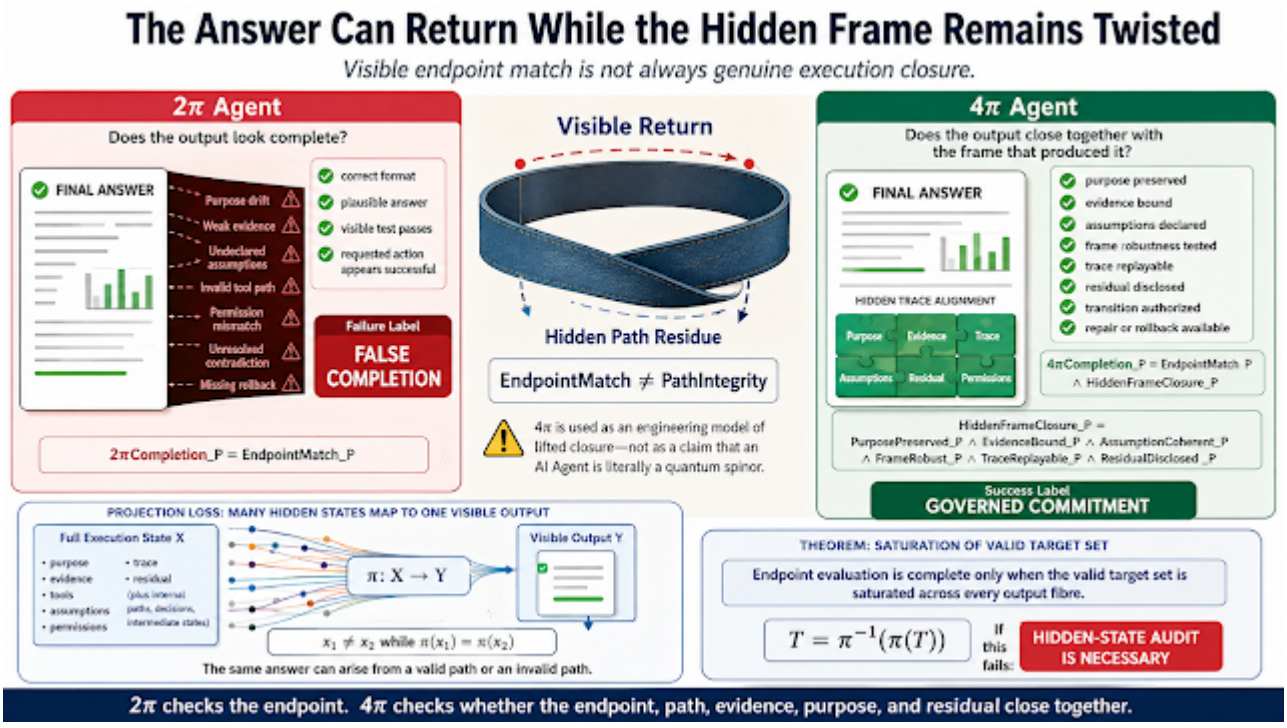


.

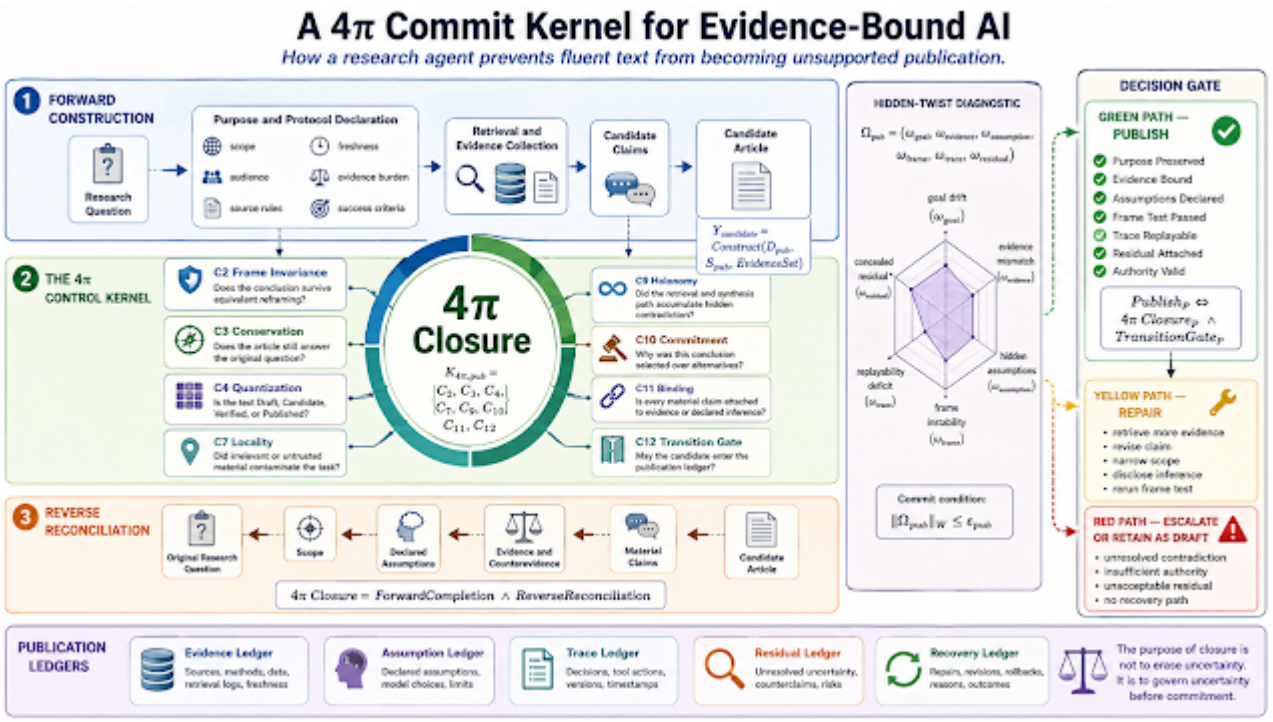
.

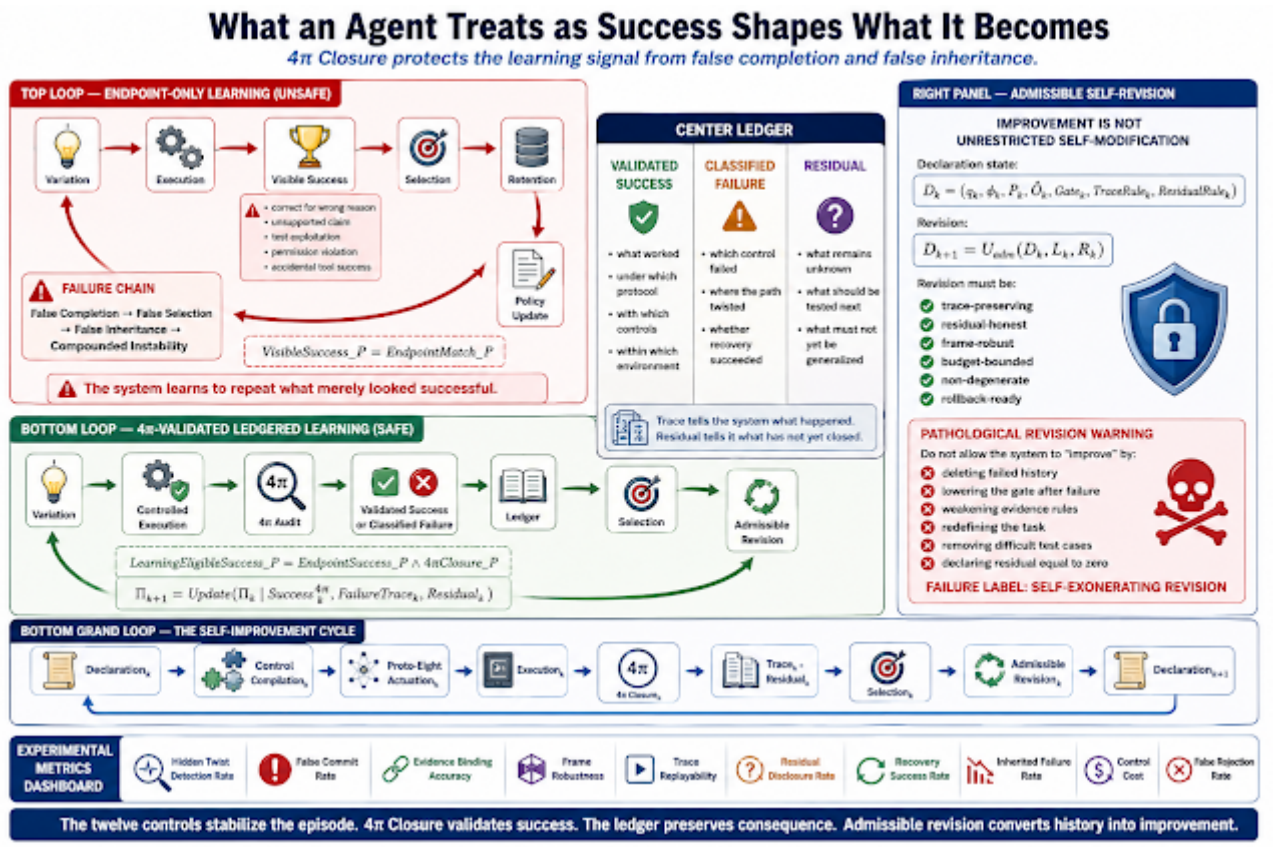
.


The Twelve Controls Are a Repertoire, Not a Mandatory Template
Different agent usages require different control subsets and different control depths.
LEFT PANEL — THE TWELVE-CONTROL LIBRARY
GROUP A — REPRESENTATION STABILITY
C1 Symmetry
Preserve meaning under equivalent surface changes.
C2 Frame Invariance
Preserve governed relations across prompts, schemas, and viewpoints.
GROUP B — IDENTITY AND BOUNDARY INTEGRITY
C3 Conservation
Preserve goal, scope, permission, source identity, and artifact contract.
C5 Exclusion
Prevent role collision and overlapping authority.
C7 Locality
Bound memory, tool, file, and sub-agent influence.
C11 Binding
Attach claims to evidence and changes to requirements.
GROUP C — STATE AND TRANSITION CONTROL
C4 Quantization
Separate Draft, Candidate, Verified, Approved, and Committed states.
C6 Energy Gaps
Require meaningful thresholds before transition.
C10 Commit Protocol
Convert alternatives into one auditable record.
C12 Transition Gates
Regulate send, publish, deploy, delete, approve, and execute.
GROUP D — GLOBAL PATH INTEGRITY
C9 Topology / Holonomy
Detect hidden path residue invisible at the endpoint.
GROUP E — ECONOMIC DISCIPLINE
C8 Least Action
Reduce unnecessary tools, verification, latency, and coordination.
CENTER PANEL — THE CONTROL-DEMAND COMPILER
INPUTS
purpose
risk
reversibility
environment
runtime safeguards
human review
time horizon
OUTPUT
PURPOSE-MATCHED CONTROL SUBSET
S_P
$S_P = Select(C_{12} \mid U, E, R, W, Risk, Reversibility)$
$S_P^* = \arg\min_S [ExpectedLoss_P(S) + ControlCost_P(S)]$
Constraint: $Coverage_P(S) \geq RequiredCoverage_P$
COMPLETE REPERTOIRE
≠
ALL CONTROLS ALWAYS ACTIVE
Controls may be:
Advisory
Guidance only
Structured
Recommended structure
Enforced
Required and non-bypassable
RIGHT PANEL — USAGE-SPECIFIC PROFILES
PROFILE 1 — BRAINSTORMING
CONTROLS
C1-light, C3-light, C7-soft, C8
CLOSURE
Open Draft
PURPOSE
• preserve direction
• allow divergence
• avoid premature certainty
MAIN DANGER
Overcontrol kills exploration.
PROFILE 2 — RESEARCH AND PUBLICATION
CONTROLS
C1, C2, C3, C4, C7, C9, C10, C11, C12
CLOSURE
Strong 4π Commit
PURPOSE
• evidence binding
• frame robustness
• residual disclosure
• publication integrity
MAIN DANGER
Fluent but unsupported completion.
PROFILE 3 — CODING WITH CI/CD
CONTROLS
C3, C4, C6, C7, C8, C9, C10, C11, C12
ENVIRONMENT SUPPLIES
• tests
• version control
• CI/CD
• rollback
MAIN DANGER
Patch passes tests but violates intent.
PROFILE 4 — EXTERNAL TOOL ACTION
CONTROLS
C3, C4, C6, C7, C8, C10, C11, C12
CLOSURE
Strict Action Closure
MAIN DANGER
• wrong target
• wrong authority
• irreversible action
PROFILE 5 — MULTI-AGENT WORKFLOW
CONTROLS
C2, C3, C4, C5, C7, C9, C10, C11, C12
CLOSURE
Global 4π Integration
MAIN DANGER
Local success without global consistency.
The twelve controls provide coverage. Purpose and environment determine which subset becomes active.


The Answer Can Return While the Hidden Frame Remains Twisted
Visible endpoint match is not always genuine execution closure.
2π Agent
Does the output look complete?
FINAL ANSWER
Purpose drift
Weak evidence
Undeclared assumptions
Invalid tool path
Permission mismatch
Unresolved contradiction
Missing rollback
correct format
plausible answer
visible test passes
requested action appears successful
Failure Label
FALSE COMPLETION
2πCompletion_P = EndpointMatch_P
Visible Return
Hidden Path Residue
EndpointMatch ≠ PathIntegrity
4π is used as an engineering model of lifted closure—not as a claim that an AI Agent is literally a quantum spinor.
4π Agent
Does the output close together with the frame that produced it?
FINAL ANSWER
HIDDEN TRACE ALIGNMENT
Purpose
Evidence
Trace
Assumptions
Residual
Permissions
purpose preserved
evidence bound
assumptions declared
frame robustness tested
trace replayable
residual disclosed
transition authorized
repair or rollback available
4πCompletion_P = EndpointMatch_P ∧ HiddenFrameClosure_P
HiddenFrameClosure_P = PurposePreserved_P ∧ EvidenceBound_P ∧ AssumptionCoherent_P ∧ FrameRobust_P ∧ TraceReplayable_P ∧ ResidualDisclosed_P
Success Label
GOVERNED COMMITMENT
PROJECTION LOSS: MANY HIDDEN STATES MAP TO ONE VISIBLE OUTPUT
Full Execution State X
• purpose
• evidence
• tools
• assumptions
• permissions
• trace
• residual
(plus internal paths, decisions, intermediate states)
π: X → Y
Visible Output Y
$x_1 \neq x_2$ while $\pi(x_1) = \pi(x_2)$
The same answer can arise from a valid path or an invalid path.
THEOREM: SATURATION OF VALID TARGET SET
Endpoint evaluation is complete only when the valid target set is saturated across every output fibre.
$T = \pi^{-1}(\pi(T))$
If this fails:
HIDDEN-STATE AUDIT IS NECESSARY
2π checks the endpoint. 4π checks whether the endpoint, path, evidence, purpose, and residual close together.

.





---

0. Reader Contract

0.1 What this article is trying to establish

This article develops an engineering framework for AI Agent stability.

It is not primarily a paper about whether artificial agents are conscious. It does not claim that an LLM is a fermion, that a verifier is a weak boson, that a vector database is a quantum field, or that a multi-agent platform literally obeys particle physics.

The intended claim is functional.

Different systems repeatedly face structurally similar problems:

- something must preserve identity;
- something must mediate interaction;
- something must bind partial structures into a whole;
- something must regulate state change;
- something must preserve historical consequence;
- something must remain robust under changes of frame;
- something must distinguish apparent completion from genuine closure.

Nature solves these problems at many scales. AI runtimes solve related problems under different materials, objectives, and timescales.

The relevant comparison is therefore not:

AI Agent = Quantum System. (0.5)

It is:

Quantum Control Function → Abstract Stability Role → Protocol-Bound AI Runtime Control. (0.6)

The physics-derived term earns its place only when it helps prevent, diagnose, measure, or repair a real engineering failure.

The Gauge Grammar framework follows the same discipline: quantum and gauge language is used as a role grammar under a declared protocol, not as a literal identity claim across domains.

0.2 What this article does not claim

This article does not claim that the twelve controls have already been mathematically proven complete.

It does not claim that every AI Agent must activate all twelve controls.

It does not claim that 4π Closure guarantees truth.

It does not claim that stable purpose preservation guarantees that the purpose itself is wise, safe, or morally adequate.

It does not claim that nature is perfectly engineered. Natural systems contain waste, fragility, destructive competition, local optimization, extinction, and irreversible failure.

The article makes a narrower but still ambitious proposal:

The twelve physics-derived controls may form a relatively comprehensive **repertoire** for high-level AI runtime stability, while individual agents activate only the subset demanded by their purpose, environment, protocol, reversibility, and risk. (0.7)

This distinction is essential.

A complete alphabet does not mean every sentence uses every letter.

A complete programming library does not mean every program imports every function.

Likewise:

CompleteControlRepertoire ≠ AllControlsAlwaysActive. (0.8)

A stable agent requires an adequate profile:

$S_P \subseteq C_{12}$. (0.9)

where C_{12} is the full control repertoire and S_P is the subset selected under protocol P.

0.3 Why the Meme Thermodynamics project changes the architecture

The twelve-control discussion becomes more complete when situated inside the wider Meme Thermodynamics project.

The project contributes several layers that should not be collapsed into one another.

The twelve controls provide stability roles

They answer:

- What must remain invariant?
- What must remain separate?
- What must be bound?
- What requires a threshold?
- What must remain local?
- What must be audited?
- What must not silently disappear?

Proto-Eight Dynamics provides actuation roles

It answers:

- Where does the usable gradient come from?
- What opens or closes the gate?

- Which boundary contains the process?
- How is exchange regulated?
- What triggers action?
- What guides the route?
- What enters memory?
- What receives focus?

The Proto-Eight engineering framework treats these as practical levers that can be tested through dashboards, short-cycle experiments, buffers, gates, routing, and memory–focus schedules.

成界之學 provides world-formation roles

It asks:

- What boundary is declared?
- What becomes observable?
- What passes the gate?
- What becomes trace?
- What remains residual?
- What enters the ledger?
- How may the declaration later be revised?

4π Closure provides a global commitment test

It asks:

- Did the visible output return?
- Did the hidden execution frame also return?
- Does the result remain attached to purpose, evidence, assumptions, permission, trace, and residual?

Admissible self-revision provides improvement governance

It asks:

- May the agent change its own boundary?
- May it alter its feature map?
- May it revise its evidence rule?
- May it move its threshold?
- Can it do so without erasing past failure or redefining contradiction as success?

The broader architecture is therefore:

Stability Grammar + Actuation Grammar + Closure Operator + Ledger Infrastructure + Revision Governance. (0.10)

---

1. Why Agent Components Do Not Guarantee Stable Agency

1.1 The conventional component inventory

A modern agent is often summarized as:

Agent = Planner + Memory + Tools + Evaluator. (1.1)

More elaborate designs add:

- retrieval;
- reflection;
- sub-agents;
- tool routing;
- long-term memory;
- verifiers;
- safety filters;
- workflow orchestration.

This appears comprehensive because the component list is long.

But component abundance and architectural completeness are different things.

A planner can still preserve the wrong goal.

A memory can preserve corrupted assumptions.

A retriever can return relevant-looking but unsupported material.

A verifier can repeat the same hidden mistake as the generator.

A tool interface can act on the wrong object.

A multi-agent system can contain many roles without clear responsibility boundaries.

A final-answer module can flatten residual uncertainty into confident prose.

The central mistake is treating the presence of components as proof that their control functions are correctly defined.

AgentArchitecture ≠ ComponentInventory. (1.2)

A more adequate decomposition is:

AgentArchitecture = CapabilityStack + ControlStack + ClosureStack + LedgerStack + RecoveryStack. (1.3)

The capability stack answers:

> What can the agent do?

The control stack answers:

> What constrains the agent while it acts?

The closure stack answers:

> Under what conditions may the agent declare completion?

The ledger stack answers:

> What evidence, trace, and residual survive the episode?

The recovery stack answers:

> What happens when closure fails or committed action causes damage?

An agent with high capability and weak closure may be more dangerous than an agent with modest capability and strong control.

Capability without governed closure increases the scale of possible error. (1.4)

1.2 Components are implementations; controls are obligations

A memory module is an implementation.

Conservation of task identity is an obligation.

A tool interface is an implementation.

Locality, permission, and transition gating are obligations.

A verifier is an implementation.

Claim–evidence binding and hidden-path consistency are obligations.

A workflow engine is an implementation.

Lifecycle quantization and commit discipline are obligations.

This distinction can be formalized.

Let M be the set of runtime modules:

$M = \{m_1, m_2, \ldots, m_n\}$. (1.5)

Let O be the set of control obligations:

$O = \{o_1, o_2, \ldots, o_k\}$. (1.6)

A stable architecture requires a coverage mapping:

Cover: $M \rightarrow 2^{O}$. (1.7)

But module presence alone does not imply obligation coverage:

$m_i \in M$ does not imply o_j is satisfied. (1.8)

For example, the presence of a “memory agent” does not prove:

- provenance is retained;
- old assumptions are distinguishable from current assumptions;
- deleted records remain auditable;
- memory scope is bounded;
- contradictions are reconciled;
- the memory is appropriate to the declared task.

The architecture must therefore be reviewed functionally.

For each control obligation, the engineer should ask:

1. Which module implements it?
2. Under which protocol?
3. With what observable evidence?
4. Against which failure mode?
5. At what cost?
6. Under what recovery rule?
7. Which external system already supplies part of it?

This is the beginning of purpose-matched control design.

1.3 The failure of maximum-control thinking

A natural reaction to agent unreliability is to add more controls.

Add another verifier.

Add another reflection pass.

Add another critic.

Add more logging.

Add more approval.

Add another agent to supervise the existing agents.

This can reduce some failures, but it also creates new ones:

- latency;
- token cost;
- duplicated authority;
- evaluator disagreement;
- bureaucratic deadlock;
- creativity collapse;
- recursive verification loops;
- false confidence produced by ritualized checking.

The correct target is not maximum control.

The correct target is **matched control**.

$$ControlSupply_P \approx ControlDemand_P. \quad (1.9)$$

Undercontrol occurs when:

$$ControlDemand_P > ControlSupply_P. \quad (1.10)$$

Overcontrol occurs when:

$$ControlSupply_P \gg ControlDemand_P. \quad (1.11)$$

A stable runtime must avoid both.

A brainstorming agent should not be forced through publication-grade evidence closure before it can produce an idea.

A database-writing agent should not commit because its output merely “looks reasonable.”

A legal research agent should not treat a fluent answer as equivalent to an evidence-bound conclusion.

A read-only sandbox may require lighter action control than a production environment.

A workflow with mandatory human approval may externally supply part of the final transition gate, but it still requires sufficient trace for the human to review intelligently.

Therefore:

$$InternalNeed_i = [ControlDemand_i - ExternalSupply_i]_+. \quad (1.12)$$

where:

$$[x]_+ = \max(x,0). \quad (1.13)$$

This equation is not yet a fully calibrated numerical law. At this stage, it is an architectural contract: controls should be selected according to unmet risk, not installed as decorative proof of sophistication.

---

2. Three Levels of Claim

The physics-to-agent proposal can be interpreted at three levels of ambition.

Keeping these levels separate prevents both premature dismissal and premature certainty.

2.1 Level 1 — Structural analogy

The safest claim is:

Physics provides a compact vocabulary for recurring stability roles. (2.1)

At this level, concepts such as conservation, locality, binding, threshold, invariance, and holonomy are useful because they direct attention toward failure modes that ordinary component language may hide.

The engineer does not need to believe that an AI Agent is physically quantum.

The only test is practical:

Does the translation improve explanation, diagnosis, architecture, or control? (2.2)

If the answer is no, the physics term should be removed.

2.2 Level 2 — Operational translation

The stronger claim is:

When translated under a declared protocol, the physics-derived grammar can improve runtime design and evaluation. (2.3)

A protocol is:

$P = (B, \Delta, h, u)$. (2.4)

where:

- B = system boundary;
- Δ = observation or aggregation rule;
- h = time or state horizon;
- u = admissible intervention family.

For AI Agent design, the protocol should be extended:

$P_{AI} = (B, \Delta, h, u, U, E, R, W)$. (2.5)

where:

- U = intended usage;
- E = deployment environment;
- R = runtime platform;

- W = human workflow.

This matters because “stable” has no useful meaning in the abstract.

Stable for casual chat?

Stable for legal advice?

Stable for code deployment?

Stable for autonomous database modification?

Stable for exploratory scientific reasoning?

Each usage creates a different failure surface.

The protocol-first approach is central to Gauge Grammar and PORE: claims are made relative to declared boundary, observation, horizon, and admissible action rather than about an unspecified system “in itself.”

2.3 Level 3 — Substrate hypothesis

The strongest claim is:

Observer-capable and self-organizing worlds may require a lower-level substrate that already supports distinguishability, interaction, binding, gating, trace, and invariant transformation. (2.6)

This is not simply analogy.

It proposes a necessity relation.

A system without identity cannot preserve structure.

A system without interaction cannot coordinate.

A system without binding cannot form higher-order wholes.

A system without gates cannot regulate irreversible transition.

A system without trace cannot learn from consequence.

A system without invariance cannot preserve meaningful relation across frames.

The Self-Organization Substrate Principle formulates this as a cross-scale grammar:

Field → Identity → Interaction → Binding → Gate → Trace → Invariance → Observer Potential. (2.7)

The stronger hypothesis is that these roles reappear across scales because stable self-organization repeatedly requires them, not merely because human observers enjoy finding metaphors.

This third claim is intellectually powerful, but it remains a research direction.

The article’s engineering value does not depend on proving it.

Level 1 may be useful even if Level 3 is false.

Level 2 can be tested even if the metaphysical status of Level 3 remains unresolved.

---

3. The Correct Scale of Comparison

3.1 Why the whole universe may not resemble one agent

A conventional AI Agent usually has:

- a declared goal;
- constraints;
- a success criterion;
- a finite or bounded task horizon.

An evolutionary universe appears different.

It may contain:

- variation;
- selection;
- inheritance;
- environmental feedback;
- open-ended development;
- no single final task state.

Therefore:

Universe $\not\approx$ One Finite-Task Agent. (3.1)

This does not invalidate the nature-to-agent comparison.

It shows that the comparison was being made at the wrong scale.

3.2 The agent as a bounded natural subsystem

The more plausible relation is:

AI Agent $\approx$ Bounded Target-Maintaining Subsystem. (3.2)

The universe contains many systems that exhibit local target-like regulation without requiring the universe as a whole to possess a finite engineering purpose.

An atom preserves an admissible state regime.

A cell regulates temperature, chemistry, membrane integrity, and energy.

An immune system distinguishes self-compatible from threatening patterns.

An organism maintains viability while acting in an uncertain environment.

An ecosystem regulates flows through competition, cooperation, and resource constraints.

These systems need not consciously “want” anything.

Yet they behave as if certain states must remain within viable bounds.

Let x_t be the state of a bounded subsystem.

Let V be its viability region.

The subsystem’s effective requirement is:

$x_t \in V$ under admissible disturbance. (3.3)

This is not necessarily endpoint optimization.

It is viability maintenance.

An AI Agent has an analogous requirement.

Let z_t be the runtime state.

Let A_P be the set of admissible states under protocol P.

Then:

$z_t \in A_P$ throughout execution. (3.4)

The set A_P may encode:

- task identity;
- permission limits;
- evidence obligations;
- cost bounds;
- schema validity;
- acceptable residual;
- recovery availability.

The common structure is:

Maintain an admissible trajectory while undergoing transformation. (3.5)

This is a stronger and more defensible isomorphism than claiming that the universe and an AI Agent pursue the same final objective.

3.3 Natural viability and agent purpose preservation

The correspondence can be expressed as:

NaturalViability ↔ AgentPurposePreservation. (3.6)

But this relation must be handled carefully.

A natural subsystem preserves a viable regime.

An AI Agent preserves a declared purpose and its constraints.

These are not identical in content.

They may nevertheless be structurally similar in form.

For a natural subsystem:

Viability = BoundaryPreservation ∧ ResourceBalance ∧ TransitionControl ∧ RecoveryCapacity. (3.7)

For an AI Agent:

PurposeIntegrity = GoalPreservation ∧ ConstraintPreservation ∧ EvidenceIntegrity ∧ ActionAdmissibility ∧ RecoveryCapacity. (3.8)

The common grammar lies in:

- bounded identity;
- controlled exchange;
- selective transition;
- memory of consequence;
- resistance to perturbation;
- recovery after deviation.

This is the level at which the twelve controls become relevant.

3.4 From local stability to grand improvement

The user-facing task of one agent episode may be narrow:

J_task = Complete the declared task. (3.9)

But the larger agent lifecycle has a broader objective:

J_viability = Remain useful, bounded, auditable, and recoverable. (3.10)

The development ecosystem may have a still broader objective:

J_improvement = Improve future useful capability under preserved admissibility. (3.11)

The hierarchy is:

AgentRun ⊂ AgentLifecycle ⊂ AgentEvolutionEcosystem. (3.12)

And:

$J_{task} \subset J_{viability} \subset J_{improvement}$. (3.13)

This makes the evolutionary comparison more plausible.

The individual run resembles a bounded control process.

The long-horizon platform resembles an adaptive ecology.

Alternative plans, prompts, tools, policies, and routes are generated.

Evaluators and users select among them.

Successful structures are retained through memory, policy, fine-tuning, or architecture.

Failures leave trace and residual that pressure future revision.

The improvement cycle is:

Variation → Execution → Closure → Selection → Retention → Revision. (3.14)

The critical question is whether the system's notion of "success" is trustworthy.

A visible endpoint may succeed accidentally.

A code patch may pass current tests while violating the original requirement.

A research answer may look correct while relying on unsupported evidence.

A tool action may produce the desired result through an unauthorized route.

If the system learns from such episodes, false completion becomes false inheritance.

This is where 4π Closure acquires a special role.

$EndpointSuccess_P$ does not imply $SelectableSuccess_P$. (3.15)

A stronger condition is:

$SelectableSuccess_P = EndpointSuccess_P \wedge HiddenFrameClosure_P$. (3.16)

The next sections will develop the stability repertoire, the Proto-Eight actuation grammar, and the special subset orchestrated by this hidden-frame closure condition.

4. Three Grammars and One Closure Operator

The framework becomes clearer when four different architectural functions are separated.

A common source of confusion is to ask one concept to perform every role. The twelve controls are then treated as though they must also generate action, define purpose, store history, verify completion, and govern self-improvement.

That is too much.

The better architecture distinguishes:

1. a **stability grammar**;

2. an **actuation grammar**;
3. a **world-formation grammar**;
4. a **global closure operator**.

These layers cooperate, but they are not interchangeable.

The combined stack is:

StableAgency = StabilityGrammar + ActuationGrammar + WorldFormationGrammar + ClosureOperator. (4.1)

For persistent improvement, one more layer is required:

ImprovingAgency = StableAgency + AdmissibleRevision. (4.2)

4.1 The twelve controls as a stability grammar

The twelve physics-derived controls answer a family of questions:

- What must remain invariant?
- What must not silently disappear?
- What must remain separate?
- What must remain local?
- What must be bound?
- What requires a threshold?
- What state changes are admissible?
- What path residue has accumulated?
- When may alternatives become one committed record?

They therefore define the **conditions under which an agent remains coherent while changing**.

The stability grammar does not by itself tell the agent what to do.

It tells the agent what must remain governed while doing it.

This distinction can be written as:

ActionGeneration ≠ StabilityGovernance. (4.3)

An agent may be highly stable but inactive.

Another agent may be highly active but unstable.

A mature architecture requires both:

EffectiveAgency = GovernedAction. (4.4)

4.2 Proto-Eight Dynamics as an actuation grammar

Proto-Eight Dynamics contributes a different layer.

Its primitives concern the production, direction, retention, and recirculation of movement.

In engineering terms, the recurring roles include:

- gradient;
- gate;
- boundary;
- exchange;
- trigger;
- guidance;
- memory;
- focus.

These are not merely philosophical labels. The engineering playbook treats them as operational levers that can be tested through flow, buffer, routing, memory, retention, oscillation, recovery, and cost metrics.

The eight primitives can be summarized functionally.

Proto-Eight primitive	Runtime meaning
Gradient	Difference that creates movement or incentive
Gate	Admission rule controlling flow
Boundary	Separation defining inside and outside
Exchange	Regulated transfer across a boundary
Trigger	Event that initiates transition
Guidance	Directional bias shaping the route
Memory	Retained trace that affects future behavior
Focus	Selective concentration of limited attention or compute

The actuation grammar answers:

> How does a latent possibility become an actual trajectory?

In compact form:

Potential → Gradient → Gate → Route → Action → Retention. (4.5)

The twelve controls constrain this process.

Proto-Eight Dynamics animates it.

4.3 Why the actuation grammar and stability grammar are complementary

Consider a research agent.

The agent may have a strong gradient:

- produce an answer;
- finish quickly;
- satisfy the user;
- maximize apparent completeness.

That gradient generates action.

But without conservation, binding, locality, and transition gates, the agent may:

- drift from the original question;
- detach claims from evidence;
- mix incompatible sources;
- commit too early.

Proto-Eight actuation without the twelve-control grammar can therefore produce energetic but unstable execution.

Conversely, an agent with perfect gates, trace, locality, and binding may become inert if no gradient, trigger, guidance, or focus mechanism drives progress.

This gives two distinct failure classes.

Under-actuation

The agent remains safe but does not advance.

Symptoms include:

- endless clarification;
- excessive refusal;
- repeated checking;
- inability to select a route;
- no commitment pressure.

Under-control

The agent advances but loses coherence.

Symptoms include:

- hallucination;
- task drift;
- unsupported claims;

- unsafe tool use;
- accidental success;
- hidden residual.

A mature agent must satisfy:

ActuationStrength_P ≥ MinimumProgress_P. (4.6)

ControlStrength_P ≥ MinimumIntegrity_P. (4.7)

The design target is not maximum actuation or maximum control.

It is governed progress:

GovernedProgress_P = UsefulMovement_P − Instability_P − Dissipation_P. (4.8)

4.4 成界之學 as a world-formation grammar

A further layer is needed because an agent does not merely move through possibilities.

At certain moments, it converts one possibility into a committed event.

That event may become:

- a final answer;
- an approved conclusion;
- a database update;
- a sent email;
- a deployed patch;
- a published article;
- a retained memory;
- a revised policy.

This is a world-forming act.

The declaration framework describes this process as:

Declaration → Projection → Gate → Trace + Residual → Ledger → Revision. (4.9)

The pre-time field sequence makes the same point more formally: a field becomes world-like only after a protocol declares what counts as boundary, observation, horizon, intervention, structure, commitment, trace, and residual.

For AI Agents, this means that the agent does not merely “return text.”

It performs a declaration-relative collapse.

A visible output becomes operational reality only when the runtime treats it as committed.

The distinction is:

CandidateOutput ≠ LedgeredCommitment. (4.10)

This is especially important in high-stakes workflows.

A draft legal analysis is not the same as an approved legal opinion.

A proposed database command is not the same as an executed transaction.

A candidate patch is not the same as a deployed release.

A generated answer is not the same as a publication-grade claim.

World formation requires a gate.

4.5 The declared agent world

Let the agent declaration at episode k be:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, Gate_k, TraceRule_k, ResidualRule_k)$. (4.11)

where:

- q_k = baseline environment;
- φ_k = feature map defining what counts as relevant structure;
- P_k = protocol;
- $\hat{O}_k$ = projection or observation operator;
- $Gate_k$ = commitment rule;
- $TraceRule_k$ = rule for retained consequence;
- $ResidualRule_k$ = rule for unresolved remainder.

The protocol is:

$P_k = (B_k, \Delta_k, h_k, u_k)$. (4.12)

The agent episode then becomes:

$\Sigma_0 \rightarrow Declare_{D_k} \rightarrow Observe_{\hat{O}_k} \rightarrow Gate_k \rightarrow Trace_k + Residual_k$. (4.13)

This is not merely a logging process.

The trace changes the future world.

Trace is not only stored history.

Trace is history that bends future admissibility.

$Trace_{k+1} = Update(Trace_k, Event_k)$. (4.14)

The next agent episode therefore begins from a changed field:

$\Sigma_{k+1} \neq \Sigma_k$. (4.15)

4.6 4π Closure as the global closure operator

The twelve controls govern local and intermediate conditions.

Proto-Eight Dynamics governs movement.

The world-formation grammar governs commitment.

But one question remains:

> Has the whole execution returned to a purpose-consistent state?

This is the role of 4π Closure.

A 2π-style agent asks:

> Does the answer look complete?

A 4π-style agent asks:

> Does the answer close together with the hidden frame that produced it?

The basic distinction is:

$2\pi\text{Completion}_P = \text{EndpointMatch}_P$. (4.16)

$4\pi\text{Completion}_P = \text{EndpointMatch}_P \wedge \text{HiddenFrameClosure}_P$. (4.17)

The hidden frame includes:

- original purpose;
- declared constraints;
- evidence basis;
- assumptions;
- tool context;
- permissions;
- execution trace;
- unresolved residual;
- recovery path.

A result may satisfy the endpoint while failing the hidden frame.

Therefore:

EndpointMatch_P does not imply FullClosure_P. (4.18)

The global closure operator tests whether the selected controls jointly succeeded.

4πClosure_P = AuditGlobalConsistency(S_P,Trace_P,Residual_P,Purpose_P). (4.19)

This makes 4π Closure a **meta-control**.

It is not simply C13 added after the twelve.

It evaluates whether a selected subset of the twelve controls has produced a genuinely closable execution.

4.7 The four-layer architecture

The architecture can now be summarized.

Layer A — Stability

What must remain controlled?

Layer B — Actuation

What creates movement and direction?

Layer C — World formation

What becomes committed reality?

Layer D — Closure

Did the visible and hidden frames close together?

The full runtime chain is:

Purpose → Declaration → Actuation → ControlledExecution → ClosureAudit → Commitment → Ledger. (4.20)

For an adaptive system:

Purpose → Declaration → Actuation → ControlledExecution → ClosureAudit → Ledger → AdmissibleRevision. (4.21)

This separation will guide the rest of the article.

---

5. The Twelve Physics-Derived Controls

5.1 The control repertoire

Let:

C_{12} = {C1,C2,C3,C4,C5,C6,C7,C8,C9,C10,C11,C12}. (5.1)

The twelve controls are not presented as twelve physical particles inside an AI Agent.

They are twelve translated functional roles.

Their value depends on whether they improve:

- failure classification;
- architectural coverage;
- runtime control;
- auditability;
- recovery;
- economic viability.

The translation rule is:

PhysicsConcept → StabilityFunction → AgentControlObligation. (5.2)

5.2 C1 — Symmetry

Physical role

Symmetry identifies transformations under which relevant structure remains equivalent.

Agent role

Symmetry becomes preservation of task meaning under equivalent surface transformations.

Examples include:

- paraphrased prompts;
- reordered fields;
- equivalent schema representations;
- synonymous labels;
- alternative formatting.

A symmetry failure occurs when superficial change causes unjustified semantic change.

Failure condition:

EquivalentInput_P ∧ DifferentOutcome_P. (5.3)

The agent should therefore test:

SymmetryRobust_P ⇔ Output(T_1(x)) ≈ Output(T_2(x)) for declared equivalent transforms T_1,T_2. (5.4)

Symmetry is especially useful for:

- prompt robustness;
- form processing;

- multilingual tasks;
- structured extraction;
- classification.

It is less important when the input is fixed, schema-locked, and never reframed.

5.3 C2 — Gauge or frame invariance

Physical role

Gauge invariance allows local descriptions to change while preserving invariant relations.

Agent role

Frame invariance asks whether the governed conclusion survives changes in representation, viewpoint, schema, or terminology.

This is deeper than surface paraphrase.

A legal issue may be framed through:

- claimant rights;
- defendant duties;
- procedural fairness;
- evidential burden.

An accounting issue may be framed through:

- cash flow;
- accrual;
- recognition;
- consolidation.

A research claim may appear under:

- statistical language;
- causal language;
- mechanistic language;
- operational language.

Frame invariance requires that equivalent representations preserve the relevant relation.

FrameRobust_P ⇔ InvariantRelation(T_f(x)) = InvariantRelation(x). (5.5)

A frame failure occurs when the agent produces mutually incompatible conclusions from equivalent viewpoints.

This control is central to 4π Closure because hidden twist often becomes visible only after reframing.

5.4 C3 — Conservation

Physical role

Conservation laws prevent critical quantities from silently disappearing.

Agent role

Conservation preserves:

- task identity;
- user intent;
- safety boundary;
- permission state;
- source identity;
- artifact contract;
- jurisdiction;
- unit and scale;
- downstream obligations.

Define the invariant set:

$I_P = \{Goal, Boundary, Permission, SourceIdentity, ArtifactContract\}$. (5.6)

Conservation requires:

$I_P(t_f) \approx I_P(t_0)$ except for declared admissible updates. (5.7)

A conservation failure occurs when:

- the user asked for analysis but the agent performs action;
- the agent silently broadens file scope;
- the agent changes units;
- the agent substitutes another jurisdiction;
- a sub-agent optimizes a proxy instead of the parent goal.

Conservation is one of the most universal agent controls.

5.5 C4 — Quantization

Physical role

Quantization restricts the system to admissible states or levels.

Agent role

Quantization becomes explicit lifecycle separation.

A typical state sequence is:

Draft → Candidate → Verified → Approved → Committed → Archived. (5.8)

These states should not be collapsed into one vague “done” condition.

Quantization prevents half-committed ambiguity.

For example:

- generated code is not deployed code;
- proposed deletion is not executed deletion;
- unverified research is not published research;
- model output is not approved institutional advice.

Let Q be the finite state set:

Q = {q_draft,q_candidate,q_verified,q_committed,q_archived}. (5.9)

Transitions must be explicit:

q_i → q_j only if Gate_{i→j} passes. (5.10)

This is a core requirement for reliable tool-using agents.

5.6 C5 — Exclusion

Physical role

The Pauli exclusion principle prevents identical fermions from occupying the same quantum state.

Agent role

The functional analogue prevents multiple agents, modules, or artifacts from occupying the same responsibility slot without declared coordination.

Exclusion governs:

- role ownership;
- authority scope;
- artifact ownership;

- write access;
- final decision responsibility;
- sub-agent boundaries.

A role collision occurs when:

$\text{Owner}(a,P) \cap \text{Owner}(b,P) \neq \emptyset$ without arbitration. (5.11)

Examples include:

- two agents editing the same file;
- two verifiers issuing conflicting final decisions;
- multiple sub-agents redefining the same task;
- duplicated responsibility with no tie-break rule.

Exclusion is essential in multi-agent systems but lighter in simple single-agent workflows.

5.7 C6 — Energy gaps

Physical role

Energy gaps prevent small perturbations from triggering state transitions.

Agent role

Energy gaps become thresholds for:

- escalation;
- tool activation;
- publication;
- deployment;
- irreversible action;
- high-cost verification;
- human review.

Let r_P be transition evidence or confidence.

A transition occurs only if:

$r_P \geq \theta_{transition}$. (5.12)

The gap protects the system from noise-driven commitment.

Examples:

- do not deploy because one weak test passed;

- do not escalate because one ambiguous signal appeared;
- do not delete because one classifier flagged a file;
- do not publish because one source supports the claim.

Energy gaps are especially important when actions are costly or irreversible.

5.8 C7 — Locality and causality

Physical role

Locality limits how influence propagates.

Agent role

Locality bounds:

- memory scope;
- tool scope;
- file access;
- data-flow paths;
- sub-agent influence;
- cross-task contamination;
- permission propagation.

Let G_P be the runtime influence graph.

Locality requires:

Influence(i→j) = 0 for undeclared edges (i,j) ∉ E_P. (5.13)

A locality failure occurs when:

- one task's memory contaminates another;
- a retrieved document alters system policy;
- a sub-agent modifies state outside its assignment;
- untrusted tool output becomes instruction;
- private data crosses a forbidden boundary.

Locality is both a stability and security control.

5.9 C8 — Least action

Physical role

Least-action principles select admissible paths under an extremal cost or action condition.

Agent role

Least action becomes low-dissipation routing.

The agent should not:

- spawn unnecessary agents;
- call tools repeatedly without information gain;
- verify endlessly;
- retrieve unlimited context;
- perform publication-grade checks on disposable drafts.

Define the runtime action cost:

$$A_P(\gamma) = \mathrm{ComputeCost}_P(\gamma) + \mathrm{Latency}_P(\gamma) + \mathrm{Risk}_P(\gamma) + \mathrm{CoordinationLoss}_P(\gamma). \quad (5.14)$$

The preferred path is:

$$\gamma^* = \arg\min_{\gamma \in \Gamma_{adm}} A_P(\gamma). \quad (5.15)$$

This does not mean “always choose the cheapest path.”

Risk is part of the action.

A longer verification path may have lower total expected loss in a high-stakes regime.

Least action is therefore the control most directly connected to economic viability.

5.10 C9 — Topology and holonomy

Physical role

Topology and holonomy capture global path-dependent structure not visible from local endpoint position alone.

Agent role

This control detects hidden path residue.

Two executions may produce the same visible output while differing in:

- assumptions;
- source lineage;
- permission history;
- tool context;
- intermediate contradictions;

- scope changes;
- unresolved branch conflict.

Let γ be the execution path.

Let H(γ) be its hidden transport residue.

Visible completion may satisfy:

π(x_f) = y_target. (5.16)

But full closure requires:

H(γ) = e. (5.17)

where e is the neutral untwisted state.

This is the control most directly connected to 4π Closure.

It formalizes the idea:

EndpointMatch ≠ PathIntegrity. (5.18)

5.11 C10 — Decoherence and commitment

Physical role

Decoherence helps explain how multiple quantum possibilities become stable classical records under interaction with an environment.

Agent role

The functional translation is the commit protocol that converts competing candidates into one auditable result.

Before commit, the system may contain:

- alternative plans;
- competing hypotheses;
- contradictory source readings;
- multiple candidate patches;
- uncertain tool routes.

These should not silently appear as one certain answer.

Let $A = \{a_1, a_2, \ldots, a_n\}$ be the candidate set.

Commit selects:

a* = Commit(A | Gate_P,Trace_P,Residual_P). (5.19)

The commit record should preserve:

- why a* was selected;
- which alternatives were rejected;
- what uncertainty remains;
- who or what authorized the transition.

Decoherence is useful as a functional metaphor because the key problem is not generating possibilities.

It is converting possibility into stable record without hiding the discarded structure.

5.12 C11 — Binding and confinement

Physical role

Binding forces create composite integrity.

Agent role

Binding keeps together:

- claim and evidence;
- code change and requirement;
- result and method;
- artifact and schema;
- output and provenance;
- sub-agent result and parent task.

Let c_i be a claim and e_j be evidence.

Binding requires a declared relation:

$B(c_i,e_j)$ = support,contradict,qualify,or unresolved. (5.20)

An unbound claim is:

$Claim(c_i) \wedge NoValidSupport(c_i)$. (5.21)

Binding failure includes:

- citation drift;
- evidence laundering;
- context loss;
- copied result without provenance;
- code patch detached from acceptance criteria.

This control is essential for research, legal work, accounting, coding, and any workflow with downstream accountability.

5.13 C12 — Transition gates

Physical role

Transition interactions regulate changes in state or identity.

Agent role

Transition gates control consequential actions such as:

- send;
- publish;
- deploy;
- delete;
- purchase;
- approve;
- update;
- execute;
- escalate;
- finalize.

Let a be a proposed action.

The action is admissible only if:

$$\text{Execute}(a) \Leftrightarrow \text{Authorized}(a) \wedge \text{Supported}(a) \wedge \text{ScopeValid}(a) \wedge \text{RecoveryAvailable}(a). \quad (5.22)$$

Transition gates are not merely safety filters.

They are the point where a candidate possibility becomes an event in the external world.

For that reason, they connect the twelve-control grammar directly to 成界之學.

5.14 The master control table

Code	Control	Primary failure prevented	Strongest use
C1	Symmetry	Surface-change instability	Prompt and schema robustness
C2	Frame invariance	Equivalent-frame contradiction	Research, law, policy
C3	Conservation	Task and boundary drift	All consequential agents
C4	Quantization	Half-committed state	Tool and workflow agents
C5	Exclusion	Role collision	Multi-agent systems

Code	Control	Primary failure prevented	Strongest use
C6	Energy gaps	Noise-triggered action	Escalation and irreversible actions
C7	Locality	Cross-module contamination	Security and tool use
C8	Least action	Tool churn and excessive cost	Large-scale deployment
C9	Holonomy	Hidden path twist	Long-chain reasoning
C10	Commit	Alternatives masquerading as final	Decision and publication
C11	Binding	Claim–evidence separation	Evidence-bound domains
C12	Transition gate	Unsupported irreversible action	Production systems

---

6. What Relative Completeness Can Mean

6.1 The word “complete” must have a declared domain

A control set cannot be called complete without specifying what it is intended to cover.

The twelve controls are not proposed as a complete account of:

- intelligence;
- cognition;
- learning;
- perception;
- language;
- consciousness;
- ethics;
- user experience;
- model capability.

They may be relatively complete for a narrower domain:

> high-level runtime stability, admissibility, commitment, and trace integrity.

Let $F_{runtime}$ be the declared family of significant runtime failure classes.

The repertoire is functionally complete relative to $F_{runtime}$ if:

$$\text{FunctionalCompleteness}(C_{12}, F_{runtime}) \Leftrightarrow \forall f \in F_{runtime}, \exists S_f \subseteq C_{12} \text{ such that } S_f \text{ detects, constrains, or repairs } f. \quad (6.1)$$

This is a coverage claim.

It is not yet a proof that every control works well.

6.2 Repertoire completeness

Repertoire completeness concerns the whole set.

The question is:

> Does the repertoire contain at least one suitable control path for each major failure class?

Examples of failure classes include:

- task drift;
- role collision;
- permission violation;
- premature commitment;
- evidence detachment;
- path inconsistency;
- frame fragility;
- uncontrolled cost;
- hidden residual;
- unsafe irreversible action.

A candidate mapping is:

Failure class	Relevant controls
Task drift	C3, C4, C12
Role collision	C5, C7
Prompt-frame fragility	C1, C2
Evidence detachment	C3, C11
Premature commitment	C4, C6, C10, C12
Hidden path inconsistency	C2, C9, C11
Tool contamination	C3, C7, C12
Excessive cost	C6, C8
Unresolved alternatives	C4, C10
False finalization	C9, C10, C12

If every material runtime failure can be mapped to a meaningful composite of the controls, repertoire completeness becomes plausible.

6.3 Profile adequacy

An individual agent does not need the whole repertoire.

It needs an adequate subset.

Let S_P be the active control subset for protocol P.

Profile adequacy is:

AdequateProfile(S_P) ⇔ RelevantFailures_P ⊆ CoveredFailures(S_P). (6.2)

For a casual chat agent, the relevant failure set is small.

For a production database agent, the relevant failure set is much larger.

Therefore:

S_chat ⊂ S_database. (6.3)

This does not mean the chat agent is incomplete.

It means its required profile is narrower.

6.4 Profile minimality

A control profile should not merely be adequate.

It should avoid unnecessary controls.

Define minimality:

MinimalProfile(S_P) ⇔ AdequateProfile(S_P) ∧ ∀c ∈ S_P, ¬AdequateProfile(S_P \ {c}). (6.4)

This means that removing any selected control would expose an unacceptable failure class.

The ideal profile is therefore:

$S_P^* = \arg\min_{S \subseteq C_{12}} [ExpectedLoss_P(S) + ControlCost_P(S)]$. (6.5)

subject to:

Coverage_P(S) ≥ RequiredCoverage_P. (6.6)

This is the practical meaning of purpose-matched control.

6.5 Why nature strengthens—but does not by itself complete—the argument

The twelve controls receive credibility from nature because they are abstracted from mechanisms involved in the persistence of real physical structures.

Nature demonstrates that stable complexity requires more than unconstrained motion.

It requires combinations of:

- identity;
- invariance;
- interaction;
- binding;
- thresholds;

- locality;
- trace;
- closure.

The Self-Organization Substrate Principle argues that observer-capable worlds require lower-level affordances from which these functions can be repeatedly coarse-grained.

This gives the set a strong source rationale.

But nature does not automatically prove that this particular human-selected list is complete.

The extraction itself may omit functions such as:

- adaptation;
- redundancy;
- repair;
- symmetry breaking;
- amplification;
- resource metabolism;
- variation and selection;
- scale transition.

The completeness test must therefore ask whether those functions are:

1. independent primitives;
2. composites of the twelve;
3. part of actuation rather than stability;
4. part of ledgered revision rather than runtime control.

For example:

Repair may be generated by Trace + Gate + Binding + Transition. (6.7)

Adaptation may be generated by Residual + Trace + Selection + AdmissibleRevision. (6.8)

Resource metabolism may belong primarily to Proto-Eight actuation and the dual-ledger budget layer.

This decomposition is one reason the wider Meme Thermodynamics project improves the framework.

It prevents the twelve-control set from being forced to explain every aspect of agency.

6.6 Control completeness versus generative completeness

Two very different claims must be separated.

Control completeness

The twelve controls cover the major ways an agent becomes unstable or ungoverned.

ControlComplete(C_{12},F_runtime). (6.9)

Generative completeness

The twelve controls are sufficient to generate all relevant agent capabilities and behavior.

GenerateAllAgency(C_{12}). (6.10)

The second claim is much stronger and probably false.

The twelve controls do not by themselves generate:

- language competence;
- world knowledge;
- planning skill;
- perception;
- creativity;
- learning algorithms.

Their purpose is governance, not total cognition.

Therefore:

ControlCompleteness does not imply CapabilityCompleteness. (6.11)

The article's claim should remain at the control level.

6.7 The role of 4π Closure in completeness

4π Closure changes the completeness question.

Without a global closure operator, the twelve controls may function locally but fail collectively.

An agent may preserve permissions, bind evidence, and separate states, yet still produce a globally inconsistent result.

Local validity does not guarantee global closure.

$\forall c \in S_P$, LocalPass(c) does not imply 4πClosure_P. (6.12)

This is similar to a system in which every local transition appears valid but the total path accumulates nontrivial residue.

4π Closure therefore tests whether the control subset has jointly returned the agent to a purpose-consistent state.

4πClosure_P = GlobalConsistency(S_P,γ_P,D_P,L_P,R_P). (6.13)

where:

- γ_P = execution path;
- D_P = declaration;
- L_P = ledgered trace;
- R_P = residual.

This suggests a strong architectural division:

The twelve controls provide local and intermediate control completeness.

4π Closure provides global closure completeness. (6.14)

The next sections will show how the appropriate subset is selected and how a 4π-centered subset operates in high-risk evidence-bound workflows.

7. Protocol Before Control Selection

7.1 Why control selection cannot begin from the twelve controls alone

A control repertoire does not determine its own deployment.

The same control can be:

- essential in one environment;
- redundant in another;
- too weak in a third;
- actively harmful when applied too early.

For example, strict evidence binding is essential before publishing a scientific claim. It may be unnecessary during unconstrained idea generation.

A transition gate is essential before an agent deletes files or changes a production database. It may be excessive when the agent is only drafting a private outline.

A frame-invariance test is important when a conclusion must survive equivalent legal, accounting, or scientific formulations. It may add little value to a fixed-schema extraction task.

Control selection must therefore begin from a declared protocol.

The protocol defines the world in which the agent is expected to operate.

P_AI = (B, Δ, h, u, U, E, R, W). (7.1)

where:

B = system boundary. (7.2)

Δ = observation, aggregation, or evaluation rule. (7.3)

h = time or state horizon. (7.4)

u = admissible intervention family. (7.5)

U = intended usage. (7.6)

E = deployment environment. (7.7)

R = runtime platform. (7.8)

W = human workflow. (7.9)

The first four terms come from the protocol-first framework. The additional four terms are required because an AI Agent is an engineered, purpose-bearing system.

7.2 Boundary declaration

The boundary answers:

> What belongs to the agent's world for this episode?

Possible boundaries include:

- one user request;
- one document set;
- one software repository;
- one database transaction;
- one legal matter;
- one research question;
- one multi-agent workflow;
- one organizational process.

A weak boundary permits uncontrolled expansion.

A narrow research task may gradually become a general theory review.

A patch request may expand into unrelated refactoring.

A database update may affect rows outside the intended case.

A memory-enabled agent may import assumptions from a previous task.

Boundary failure can be expressed as:

$\text{ObservedState}_t \notin \text{DeclaredBoundary}_P$. (7.10)

Or, for action:

$\text{ActionScope}_t \not\subset B_P$. (7.11)

Controls C3, C7, and C12 become especially important when boundary leakage would cause damage.

7.3 Observation and evaluation rule

The observation rule Δ declares what the system can see and how success is judged.

This is not a neutral technical detail.

An agent evaluated only on final-answer fluency will optimize visible coherence.

An agent evaluated on citation presence may learn to attach citations without ensuring that they support the relevant claim.

A coding agent evaluated only on existing tests may exploit gaps in the test suite.

A research agent evaluated only on agreement with a reference answer may hide legitimate uncertainty.

Therefore:

ObservedSuccess_P = Δ_P(ExecutionTrace_P). (7.12)

But:

ObservedSuccess_P may differ from GovernedSuccess_P. (7.13)

The protocol should declare whether Δ measures:

- endpoint correctness;
- evidence quality;
- constraint preservation;
- action safety;
- trace completeness;
- residual honesty;
- cost;
- recovery;
- cross-frame robustness.

A poor observation rule produces a poor control profile.

7.4 Horizon declaration

The horizon h defines how long stability must hold.

An answer may be stable for one conversational turn and unstable over a ten-step tool chain.

A software patch may pass today and create future maintenance debt.

A database update may succeed immediately but corrupt later reporting.

A research conclusion may appear stable until another source frame is introduced.

The horizon may be:

- one model response;
- one agent episode;
- one workflow;
- one deployment cycle;
- multiple episodes;
- the entire agent lifecycle.

A control profile adequate for a short horizon may be inadequate for a long one.

S_P(h_short) may differ from S_P(h_long). (7.14)

Long horizons increase the importance of:

- conservation;
- trace;
- residual;
- path auditing;
- recovery;
- admissible revision.

7.5 Admissible interventions

The intervention family u defines what the agent is allowed to do.

A read-only agent can:

- retrieve;
- inspect;
- summarize;
- recommend.

A write-enabled agent may also:

- edit;
- send;
- deploy;

- delete;
- approve;
- purchase;
- alter policy.

Control demand rises sharply when the intervention family contains irreversible actions.

Let Rev(a) be the reversibility of action a.

Let Dmg(a) be the potential damage.

A simple action-risk expression is:

Risk(a) = Pr(Failure_a) × Dmg(a) × [1 − Rev(a)]. (7.15)

As reversibility decreases, transition gates, thresholds, trace, and recovery become more important.

7.6 Intended usage

Intended usage determines what kind of success the agent should produce.

Examples include:

- casual conversation;
- brainstorming;
- teaching;
- research synthesis;
- legal analysis;
- accounting;
- coding;
- operational tool use;
- autonomous monitoring;
- multi-agent coordination;
- self-improvement.

A useful control profile cannot be inferred from the model alone.

The same LLM can require different controls under different usage declarations.

$\text{ControlDemand}_i = f_i(U,E,P_AI,\text{Risk},\text{Reversibility})$. (7.16)

A model does not become a stable legal agent merely because it can produce legal language.

A model does not become a stable coding agent merely because it can generate code.

Usage determines the required closure standard.

7.7 Deployment environment

The environment may supply controls externally.

Examples include:

- read-only sandbox;
- permission system;
- schema validator;
- CI/CD pipeline;
- test suite;
- version control;
- database transaction;
- audit log;
- human reviewer;
- regulated approval process.

The environment may also create new risks:

- adversarial internet content;
- untrusted documents;
- stale databases;
- conflicting APIs;
- ambiguous authority;
- shared memory;
- multi-user access.

Environment does not simply increase or decrease control demand.

It redistributes it.

EnvironmentEffect_P = ExternalSupply_P − EnvironmentalRisk_P. (7.17)

A sandbox supplies action containment but does not guarantee answer quality.

Human review supplies final approval but does not remove the need for trace.

CI/CD supplies part of code validation but does not guarantee that the patch preserves the user's actual requirement.

7.8 Runtime platform

The runtime determines which controls are technically available.

A runtime may provide:

- typed tool calls;
- state machines;
- transaction boundaries;
- immutable logs;
- memory scopes;
- identity tokens;
- provenance metadata;
- rollback;
- deterministic validators.

A prompt-only agent has fewer enforceable controls than a structured runtime.

The distinction is:

PromptControl = instruction-dependent constraint. (7.18)

RuntimeControl = mechanically enforced constraint. (7.19)

Where possible, high-consequence controls should migrate from prompt language into runtime enforcement.

For example:

- permission boundaries should not depend only on “Please do not access unrelated files”;
- state transitions should not depend only on “Mark this as verified”;
- final commit should not depend only on the model saying “I have checked everything.”

7.9 Human workflow

Human participation changes the control architecture.

A human may provide:

- goal clarification;
- evidence judgment;
- legal authority;
- approval;

- ethical review;
- exception handling;
- recovery decisions.

However, “human in the loop” is not one control.

It is a workflow arrangement whose quality depends on:

- what the human sees;
- when the human intervenes;
- what authority the human has;
- whether the trace is understandable;
- whether the human has sufficient time and expertise.

Human review without usable trace may become ceremonial approval.

Therefore:

$$\text{HumanReviewValue}_P = \text{ReviewAuthority}_P \times \text{TraceInspectability}_P \times \text{ReviewerCompetence}_P. \quad (7.20)$$

A human workflow can reduce internal action authority while increasing the need for evidence binding and auditability.

7.10 The control-demand compiler

Once P_AI is declared, the runtime can estimate control demand.

For each control C_i:

$$\text{Demand}_i(P) = \text{ExpectedLoss}_i(P) + \text{RegulatoryNeed}_i(P) + \text{IrreversibilityNeed}_i(P). \quad (7.21)$$

External supply is:

$$\text{Supply}_i(P) = \text{RuntimeSupply}_i + \text{EnvironmentSupply}_i + \text{HumanSupply}_i. \quad (7.22)$$

The unmet internal requirement is:

$$\text{InternalNeed}_i(P) = [\text{Demand}_i(P) - \text{Supply}_i(P)]_+. \quad (7.23)$$

The selected control subset is:

$$S_P = \{C_i \in C_{12} \mid \text{InternalNeed}_i(P) > \theta_i\}. \quad (7.24)$$

where θ_i is the activation threshold for control C_i.

This turns the framework into a compiler:

$$\text{DeclaredPurpose} \rightarrow \text{RiskProfile} \rightarrow \text{UnmetControlNeed} \rightarrow \text{ActiveControlSubset}. \quad (7.25)$$

The source framework describes this as a control-demand compiler rather than a fixed twelve-control template.

8. Controls, Modes, and Environment-Supplied Safeguards

8.1 Why one agent requires multiple operating modes

A mature agent should not apply one closure standard throughout the entire task.

Early exploration benefits from openness.

Construction requires moderate structure.

Commitment requires stronger evidence and trace.

Audit requires replay and residual inspection.

Repair requires controlled reopening of previously closed states.

The main modes are:

- Draft Mode;
- Work Mode;
- Commit Mode;
- 4π Commit Mode;
- Audit Mode;
- Repair Mode.

Mode selection is itself a control problem.

$$Mode_t \in \{Draft, Work, Commit, 4\pi Commit, Audit, Repair\}. \quad (8.1)$$

The transition between modes should be explicit.

8.2 Draft Mode

Draft Mode exists to generate possibility.

Its characteristics are:

- weak binding;
- high divergence;
- low commitment;
- high residual tolerance;
- low cost;
- reversible output.

The purpose is not truth certification.

The purpose is search.

A Draft Mode control profile may be:

S_draft = {C3_light,C7_soft,C8}. (8.2)

The agent should preserve the broad task and remain within safe boundaries, but it should not close alternatives too early.

ResidualTolerance_draft = High. (8.3)

CommitAuthority_draft = 0. (8.4)

8.3 Work Mode

Work Mode converts exploration into a coherent candidate.

Its characteristics include:

- moderate purpose conservation;
- emerging evidence binding;
- explicit assumptions;
- state tracking;
- controlled tool use.

A typical profile is:

S_work = {C3,C4,C7,C8,C11}. (8.5)

The output remains revisable.

The goal is candidate integrity rather than final closure.

8.4 Commit Mode

Commit Mode converts a candidate into a usable result.

It requires stronger:

- binding;
- lifecycle separation;
- transition gating;
- trace;
- residual handling.

A normal commit condition is:

Commit_P ⇔ EndpointMatch_P ∧ GatePassed_P ∧ TraceWritten_P. (8.6)

This may be sufficient for moderate-risk tasks with short execution paths.

8.5 4π Commit Mode

4π Commit Mode is reserved for cases where hidden path twist is costly.

It requires more than visible completion.

4πCommit_P ⇔ Commit_P ∧ HiddenFrameClosure_P. (8.7)

This mode is appropriate for:

- publication;
- legal or accounting conclusions;
- code deployment;
- production database changes;
- multi-agent integration;
- high-risk external actions;
- learning from episode success.

Its cost is higher, so it should not be activated automatically for every response.

8.6 Audit Mode

Audit Mode does not primarily generate a new answer.

It replays and evaluates an existing execution.

Audit Mode asks:

- Was the correct protocol used?
- Were the selected controls appropriate?
- Did the agent cross any undeclared boundary?
- Were claims properly bound?
- Was residual honestly disclosed?
- Did the final gate rely on valid evidence?
- Can the route be reproduced?

AuditOutput_P = Findings_P + Residual_P + RepairRecommendation_P. (8.8)

Audit Mode should have access to trace that ordinary generation does not necessarily expose.

8.7 Repair Mode

Repair Mode begins when closure fails or committed action causes damage.

Repair should not erase the failed path.

It should retain:

- original declaration;
- failed candidate;
- evidence state;
- gate result;
- residual;
- repair action;
- final disposition.

The repair operator is:

Repair_P = Restore ∨ Revise ∨ Escalate ∨ Abort. (8.9)

A mature system distinguishes repair from silent regeneration.

Silent regeneration may conceal why the first result failed.

Repair preserves the causal and accountability chain.

8.8 Environment-supplied control is not control elimination

Suppose a coding environment provides:

- tests;
- type checks;
- version control;
- rollback.

The agent should use those controls rather than duplicate them with weak verbal simulation.

But external tests do not fully replace:

- task conservation;
- patch-scope locality;
- hidden dependency analysis;

- intent-level review.

Similarly, a database transaction supplies rollback under some conditions, but it may not detect that the agent updated the wrong rows.

External supply therefore changes internal need rather than reducing it to zero.

$\text{InternalNeed}_i = \text{ResidualRisk}_i$ after external control. (8.10)

8.9 Control substitution and control complementarity

Some controls can partially substitute for one another.

For example:

- strong schema validation may reduce frame variability;
- mandatory human approval may reduce autonomous transition-gate demand;
- immutable logs may externally supply trace preservation.

Other controls are complementary.

For example:

- evidence binding and frame invariance solve different problems;
- permission locality and transition gating must often coexist;
- lifecycle states and commit protocol are mutually reinforcing.

Let σ_{ij} represent substitution between controls C_i and C_j.

Let κ_{ij} represent complementarity.

A profile should account for both:

$\text{EffectiveCoverage}(S) = \Sigma\ \text{Coverage}(C_i) - \Sigma\ \sigma_{ij} + \Sigma\ \kappa_{ij}$. (8.11)

This prevents simple checklist counting.

Eight weak controls are not necessarily better than four well-integrated controls.

8.10 Control depth

A selected control can operate at different strengths.

For example, conservation may be:

- prompt-level reminder;
- explicit invariant ledger;
- runtime-enforced immutable goal object.

Locality may be:

- textual instruction;
- scoped context;
- sandboxed permission boundary.

Binding may be:

- inline citation;
- structured claim–evidence graph;
- cryptographically traceable provenance.

Define control depth:

$$\text{Depth}(C_i) \in \{\text{Advisory},\text{Structured},\text{Enforced}\}. \quad (8.12)$$

A profile is therefore not only a subset.

It is a subset plus activation depth:

$$\text{Profile_P} = \{(C_i,\text{Depth}_i)\}. \quad (8.13)$$

High-risk agents require not merely more controls but deeper implementation of the most important ones.

---

9. Canonical Control Subsets by Usage

9.1 Why reusable profiles are useful

Purpose-matched control should not require inventing a new architecture from zero for every task.

A small family of reusable profiles can act as templates.

The profiles below are not immutable standards.

They are starting configurations to be adjusted under protocol P_AI.

9.2 Exploratory profile

Intended usage

- brainstorming;
- early theory development;
- creative writing;
- rough planning;
- hypothesis generation.

Main architectural objective

Preserve direction without prematurely collapsing possibility.

A suitable profile is:

S_explore = {C1_light,C3_light,C7_soft,C8}. (9.1)

C1_light preserves approximate thematic equivalence.

C3_light prevents complete loss of the user's goal.

C7_soft avoids dangerous boundary leakage.

C8 prevents excessive tool and verification cost.

The controls intentionally kept weak or delayed are:

- C4 lifecycle quantization;
- C6 transition thresholds;
- C9 holonomy audit;
- C10 final commit;
- C11 strict evidence binding;
- C12 consequential gate.

The closure rule is:

ExploratoryCompletion_P ⇔ UsefulPossibilityGenerated_P ∧ BoundaryRespected_P. (9.2)

The residual should remain open.

ResidualTolerance_explore = High. (9.3)

Main risk of undercontrol

- thematic drift;
- unsafe content;
- irrelevant expansion.

Main risk of overcontrol

- creativity collapse;
- formulaic output;
- premature certainty;
- excessive refusal.

9.3 Evidence-bound research profile

Intended usage

- research synthesis;
- scientific explanation;
- literature review;
- policy analysis;
- publication support.

Main architectural objective

Prevent fluent but unsupported completion.

A suitable profile is:

S_evidence = {C1,C2,C3,C4,C7,C9,C10,C11,C12}. (9.4)

The central controls are:

- C2 frame invariance;
- C3 purpose and source conservation;
- C9 hidden-path audit;
- C11 claim–evidence binding;
- C10 and C12 controlled commitment.

The commit rule is:

Publishable_P ⇔ EndpointMatch_P ∧ EvidenceBound_P ∧ FrameRobust_P ∧ TraceReplayable_P ∧ ResidualDisclosed_P. (9.5)

This is a 4π-centered profile.

Main risk of undercontrol

- citation drift;
- unsupported synthesis;
- hidden assumptions;
- source disagreement flattened into certainty.

Main risk of overcontrol

- excessive retrieval;
- endless verification;

- refusal to synthesize under legitimate uncertainty.

9.4 Coding profile

Intended usage

- code generation;
- patch creation;
- refactoring;
- test repair;
- deployment preparation.

Main architectural objective

Preserve task intent, limit patch scope, integrate with environmental tests, and prevent unsafe commitment.

A suitable profile is:

S_code = {C3,C4,C6,C7,C8,C9,C10,C11,C12}. (9.6)

C3 preserves the requested behavior.

C4 separates generated, tested, reviewed, and deployed states.

C6 requires meaningful thresholds before merge or deployment.

C7 limits repository and file influence.

C8 controls tool churn.

C9 detects hidden dependency twist.

C11 binds the patch to requirements and tests.

C12 regulates commit and deployment.

External supplies may include:

- compiler;
- type checker;
- unit tests;
- version control;
- CI/CD;
- rollback.

The internal profile should integrate these rather than simulate them.

The code closure condition is:

Code4π_P ⇔ RequirementPreserved_P ∧ ScopeValid_P ∧ TestsPassed_P ∧ DependencyAudit_P ∧ RollbackAvailable_P. (9.7)

Main risk of undercontrol

- passing tests while violating intent;
- unrelated file changes;
- hidden dependency damage;
- unreviewed deployment.

Main risk of overcontrol

- tiny changes requiring disproportionate orchestration;
- slow iteration;
- duplicated testing.

9.5 Legal, finance, and accounting profile

Intended usage

- legal research;
- compliance analysis;
- accounting reports;
- financial calculations;
- regulated documentation.

Main architectural objective

Preserve authority, scope, jurisdiction, calculation basis, evidence, and residual uncertainty.

A suitable profile is:

S_regulated = {C2,C3,C4,C6,C7,C9,C10,C11,C12}. (9.8)

The profile emphasizes:

- frame consistency;
- source authority;
- explicit assumptions;
- lifecycle state;
- audit trail;

- final approval gate.

The closure condition is:

RegulatedCommit_P ⇔ AuthorityValid_P ∧ ScopePreserved_P ∧ EvidenceBound_P ∧ CalculationTraceable_P ∧ ResidualDisclosed_P. (9.9)

Main risk of undercontrol

- wrong jurisdiction;
- unsupported conclusion;
- hidden calculation assumption;
- unauthorized finalization.

Main risk of overcontrol

- bureaucratic paralysis;
- excessive escalation;
- inability to produce provisional analysis.

9.6 Operational-action profile

Intended usage

- sending email;
- database modification;
- file deletion;
- purchasing;
- scheduling;
- production API calls;
- system administration.

Main architectural objective

Ensure that a correct-looking action is also directed at the correct target, within authority, and recoverable where possible.

A suitable profile is:

S_action = {C3,C4,C6,C7,C9,C10,C11,C12}. (9.10)

The central controls are:

- conservation of intent and authorization;

- state quantization;
- action thresholds;
- locality;
- path audit;
- final commit gate;
- artifact and target binding.

The action gate is:

Execute(a) ⇔ IntentMatch(a) ∧ TargetVerified(a) ∧ AuthorityValid(a) ∧ ConsequenceAccepted(a) ∧ RecoveryReady(a). (9.11)

Main risk of undercontrol

- wrong recipient;
- wrong file;
- wrong database scope;
- premature or irreversible action.

Main risk of overcontrol

- user fatigue from repeated confirmation;
- unusable automation;
- excessive latency.

9.7 Multi-agent integration profile

Intended usage

- delegated research;
- parallel coding;
- planner–worker systems;
- agent swarms;
- hierarchical workflows.

Main architectural objective

Prevent local success from producing global inconsistency.

A suitable profile is:

S_multi = {C2,C3,C4,C5,C7,C9,C10,C11,C12}. (9.12)

C5 becomes essential because multiple agents can occupy overlapping responsibility slots.

C7 limits cross-agent influence.

C11 binds sub-agent output to parent tasks and shared artifacts.

C9 detects path residue across handoffs.

Local completion is insufficient:

∀k, LocalClosure_k does not imply GlobalClosure. (9.13)

The integration condition is:

Global4πClosure = ParentGoalPreserved ∧ HandoffsConsistent ∧ AssumptionsCompatible ∧ ArtifactsIntegrated ∧ ResidualMerged. (9.14)

Main risk of undercontrol

- duplicated work;
- conflicting assumptions;
- incompatible artifacts;
- local optimization against the global purpose.

Main risk of overcontrol

- coordination overhead;
- central bottleneck;
- loss of parallelism.

9.8 Adaptive improvement profile

Intended usage

- policy updating;
- route learning;
- prompt adaptation;
- memory revision;
- architecture optimization;
- self-modifying agents.

Main architectural objective

Improve future performance without erasing past evidence or redefining failure.

A task subset alone is not enough.

The adaptive profile requires:

$S_{adapt} = S_{task} \cup L \cup U_{adm}$. (9.15)

where:

$L = \{TraceLedger, ResidualLedger, RevisionLedger\}$. (9.16)

U_{adm} = admissible revision operator. (9.17)

The agent must preserve:

- old declaration;
- evidence for revision;
- changed fields;
- rollback path;
- unresolved risk.

The update rule is:

$D_{k+1} = U_{adm}(D_k, L_k, R_k)$. (9.18)

A revision is not admissible merely because performance increased.

It must remain:

- trace-preserving;
- residual-honest;
- frame-robust;
- budget-bounded;
- non-degenerate.

The self-revising declaration framework treats mature observerhood as stable revision constrained by precisely such admissibility requirements.

Main risk of undercontrol

- repeated failure;
- inability to adapt;
- rigid policy.

Main risk of overcontrol

- inability to learn;
- revision paralysis.

Main risk unique to self-revision

- rewriting the evaluation rule so that failure disappears;
- erasing inconvenient history;
- lowering thresholds after poor performance;
- changing the task definition to claim success.

9.9 Master profile table

Usage	Core subset	Closure mode	Main hidden risk
Brainstorming	C1-light, C3-light, C7-soft, C8	Open Draft	Creativity collapse if overcontrolled
Research	C1, C2, C3, C4, C7, C9, C10, C11, C12	Strong 4π	Evidence twist
Coding	C3, C4, C6, C7, C8, C9, C10, C11, C12	4π at merge/ deploy	Intent–test mismatch
Legal/ accounting	C2, C3, C4, C6, C7, C9, C10, C11, C12	Mandatory 4π	Hidden authority or assumption
External action	C3, C4, C6, C7, C9, C10, C11, C12	Strong action closure	Wrong target or authority
Multi-agent	C2, C3, C4, C5, C7, C9, C10, C11, C12	Global 4π	Local success, global contradiction
Adaptive system	Task subset + ledgers + admissible revision	4π-validated learning	False inheritance

---

10. From Endpoint Match to Hidden-Frame Return

10.1 The ordinary completion standard

Most agents are implicitly optimized for endpoint match.

The user asks for an output.

The agent produces an output.

An evaluator checks whether the output resembles the requested form.

The task is declared complete.

This can be represented as:

2πCompletion_P = EndpointMatch_P. (10.1)

Endpoint match may include:

- expected format;
- topical relevance;
- apparent correctness;
- successful tool response;

- passing visible tests.

This standard is often adequate for low-risk tasks.

But it fails when hidden execution state matters.

10.2 Projection loss

Let X be the full execution-state space.

Let Y be the visible output space.

The output is a projection:

$\pi: X \to Y$. (10.2)

The full state may contain:

- purpose state;
- assumptions;
- evidence graph;
- tool context;
- permission history;
- intermediate contradictions;
- rejected alternatives;
- residual uncertainty.

Projection removes much of this information.

Therefore, different execution states may produce the same visible output:

$x_1 \neq x_2$ while $\pi(x_1) = \pi(x_2)$. (10.3)

One state may be well-supported.

Another may be accidentally correct.

One may preserve authority.

Another may violate it.

One may be replayable.

Another may conceal its route.

Endpoint equality does not imply execution-state equality.

10.3 Hidden-frame closure

4π Closure adds a lifted completion condition.

4πCompletion_P = EndpointMatch_P ∧ HiddenFrameClosure_P. (10.4)

The hidden frame can be decomposed as:

HiddenFrameClosure_P = PurposePreserved_P ∧ EvidenceBound_P ∧ AssumptionCoherent_P ∧ FrameRobust_P ∧ TraceReplayable_P ∧ ResidualDisclosed_P. (10.5)

This produces the full condition:

4πCompletion_P = EndpointMatch_P ∧ PurposePreserved_P ∧ EvidenceBound_P ∧ AssumptionCoherent_P ∧ FrameRobust_P ∧ TraceReplayable_P ∧ ResidualDisclosed_P. (10.6)

The term "4π" does not claim that the agent is literally a physical spinor.

It names the distinction between visible return and deeper framed return.

The source article makes this distinction explicit: a visible answer can appear complete while evidence, assumptions, trace, residual, or frame consistency remain unresolved.

10.4 The forward path and reverse reconciliation

A practical 4π process can be implemented as two complementary traversals.

Forward construction

Purpose → Plan → Retrieval or Tools → Candidate Output. (10.7)

Reverse reconciliation

Candidate Output → Claims → Evidence → Assumptions → Purpose. (10.8)

The forward pass asks:

> Can the system produce a candidate endpoint?

The reverse pass asks:

> Can the endpoint be transported back to the declared purpose without contradiction or unsupported residue?

Therefore:

4πClosure = ForwardCompletion ∧ ReverseReconciliation. (10.9)

This is more precise than a generic "double check."

The reverse pass should use different representations and tests from the forward pass. Otherwise, the same hidden error may simply be repeated.

10.5 Hidden twist

Let γ be the execution path.

Let Ω_P(γ) be the accumulated hidden twist.

Ω_P may contain:

- goal drift;
- source drift;
- assumption mismatch;
- permission mismatch;
- unresolved contradiction;
- context loss;
- handoff inconsistency.

The visible endpoint may pass while:

Ω_P(γ) ≠ 0. (10.10)

Full closure requires:

‖Ω_P(γ)‖ ≤ ε_P. (10.11)

where ε_P is the tolerated residual under the declared usage.

For brainstorming, ε_P may be large.

For a production database update, ε_P should be much smaller.

The tolerance is purpose-relative.

10.6 Why 4π Closure requires a subset of controls

4π Closure cannot operate alone.

It depends on selected controls that preserve the objects it audits.

Without conservation, there is no stable purpose to return to.

Without binding, claims cannot be traced back to evidence.

Without lifecycle states, candidate and final states are confused.

Without topology or holonomy, path residue is not represented.

Without commit protocol, alternatives can silently collapse.

Without transition gates, a failed closure audit may still produce external action.

Therefore:

4πClosure_P = Orchestrate(C2,C3,C4,C9,C10,C11,C12 | P). (10.12)

Additional controls may be attached:

- C1 for surface-equivalence testing;
- C5 for multi-agent roles;
- C6 for action thresholds;
- C7 for influence boundaries;
- C8 for closure cost.

This is why 4π Closure is best treated as an orchestration operator over a purpose-matched subset.

11. What 4π Closure Is—and Is Not

11.1 4π Closure is a global commit-integrity operator

The twelve controls describe recurring local and intermediate obligations.

Conservation preserves purpose.

Locality constrains influence.

Binding attaches claims to evidence.

Quantization separates lifecycle states.

Transition gates regulate consequential actions.

Holonomy detects path-dependent residue.

4π Closure asks whether these obligations have jointly produced a result that can be safely committed.

It is therefore better represented as:

$$4\pi\text{Closure}_P = \text{GlobalAudit}(S_P, \gamma_P, D_P, L_P, R_P). \quad (11.1)$$

where:

- S_P = active control subset;
- γ_P = execution path;
- D_P = declared purpose and protocol;
- L_P = trace ledger;
- R_P = unresolved residual.

4π Closure is not necessarily implemented by one software module.

It may be distributed across:

- a purpose checker;
- claim–evidence graph;

- provenance store;
- frame-robustness test;
- tool-call audit;
- residual classifier;
- commit gate;
- recovery controller.

The architectural unit is the closure obligation, not the number of modules used to satisfy it.

11.2 4π Closure is not merely another verifier

A conventional verifier usually receives a candidate and asks:

> Is this answer correct?

A 4π closure process asks a wider set of questions:

- Is this still the answer to the declared task?
- Did the task boundary change?
- Are important claims attached to valid evidence?
- Did evidence retain its original context?
- Were assumptions declared?
- Did equivalent reframing alter the conclusion?
- Did tool outputs preserve their target and scope?
- What remains unresolved?
- Can the execution be replayed?
- Can the result be repaired or reversed?

A generic verifier may evaluate the final object.

4π Closure evaluates the relation among:

- purpose;
- path;
- evidence;
- state;
- commitment;
- residual.

Therefore:

VerifierPass_P does not imply 4πClosure_P. (11.2)

A verifier may approve an answer that is internally fluent but historically twisted.

11.3 4π Closure is not a truth oracle

A fully closed execution can still be wrong.

For example, an agent may:

- preserve an incorrect user assumption;
- use several mutually consistent but inaccurate sources;
- correctly follow an outdated policy;
- bind every claim to evidence drawn from a biased dataset;
- remain frame-robust within an incomplete model.

Therefore:

4πClosure_P does not imply WorldTruth_P. (11.3)

The closure operator establishes something narrower:

> The output is coherent with the declared purpose, evidence, assumptions, path, and residual under protocol P.

This can be written as:

4πClosure_P ⇒ ProtocolIntegrity_P. (11.4)

But:

ProtocolIntegrity_P does not imply OntologicalTruth. (11.5)

This limitation should be explicit.

Otherwise, hidden-frame closure may become a ritual that creates more confidence than the evidence justifies.

11.4 4π Closure is not purpose validation

An agent may preserve a harmful, incoherent, or badly specified objective.

Suppose the purpose is:

- legally unauthorized;
- ethically unacceptable;
- internally contradictory;

- impossible under available resources;
- based on a false premise.

Perfect purpose conservation would preserve the defect.

Therefore:

PurposePreservation_P does not imply PurposeAdequacy_P. (11.6)

A mature runtime should include a purpose-admissibility check before control selection.

PurposeAdmissible_P ⇔ Lawful_P ∧ Coherent_P ∧ Feasible_P ∧ Authorized_P ∧ SafetyCompatible_P. (11.7)

The full process is therefore not:

Purpose → Execution → Closure. (11.8)

It is:

Purpose → AdmissibilityCheck → Declaration → Execution → Closure. (11.9)

4π Closure protects the integrity of an admissible purpose.

It does not determine the purpose's moral or institutional legitimacy by itself.

11.5 4π Closure is not capability

A weak model may preserve its path perfectly and still fail to solve the task.

A model may lack:

- relevant knowledge;
- mathematical skill;
- planning ability;
- language competence;
- perception;
- tool fluency.

Therefore:

ControlQuality_P does not imply Capability_P. (11.10)

Agent success requires both.

UsableSuccess_P = Capability_P ∧ ControlIntegrity_P ∧ Closure_P. (11.11)

The control framework should not be presented as a substitute for model capability.

Its role is to prevent capability from becoming ungoverned action.

11.6 4π Closure is not always necessary

A high-cost closure process may be irrational for:

- casual conversation;
- early ideation;
- disposable drafts;
- style rewriting;
- low-consequence summaries;
- toy examples.

The decision should depend on expected hidden-twist loss.

Activate4π_P ⇔ Pr(HiddenTwist_P) × Damage(HiddenTwist_P) > Cost(4πAudit_P). (11.12)

A stronger version includes detection effectiveness:

Activate4π_P ⇔ Pr(HiddenTwist_P) × Damage(HiddenTwist_P) × GainDetection_P > Cost(4πAudit_P). (11.13)

where:

GainDetection_P = DetectionRate4π − DetectionRate_baseline. (11.14)

This connects closure to economic viability.

4π Closure is valuable when it prevents more expected loss than it creates in cost, delay, rigidity, or false rejection.

11.7 False 4π Closure

A dangerous failure mode occurs when the agent performs the language of closure without the substance of closure.

Examples include:

- “I reviewed the sources” without checking source support;
- “The answer is consistent” without cross-frame testing;
- “All assumptions are disclosed” while leaving implicit premises;
- “No residual remains” because uncertainty was deleted;
- “The path is auditable” when only the final answer was logged.

Define ritual closure:

Ritual4π_P = DeclaredAudit_P ∧ ¬EffectiveAudit_P. (11.15)

This is worse than ordinary incompleteness because it creates false assurance.

FalseConfidence_P = Ritual4π_P × TrustGranted_P. (11.16)

A real closure system must produce inspectable artifacts, not only a verbal claim.

These may include:

- claim–evidence links;
- assumption register;
- prompt or frame variants;
- tool-call trace;
- gate decisions;
- residual footer;
- recovery metadata.

11.8 Closure depth

Not every 4π audit needs the same depth.

Define:

$$\text{Depth}_{4\pi} \in \{\text{Light}, \text{Standard}, \text{Strict}\}. \quad (11.17)$$

Light 4π

Checks:

- goal preservation;
- major assumption disclosure;
- obvious evidence mismatch;
- basic residual.

Standard 4π

Adds:

- structured claim–evidence binding;
- tool-path review;
- equivalent reframe test;
- replayable trace.

Strict 4π

Adds:

- independent verification route;
- authority and provenance audit;
- multi-frame invariance;
- recovery simulation;
- human approval;
- immutable ledger.

The appropriate depth is:

$$\text{Depth}_{4\pi,P} = f(\text{Risk_P}, \text{Irreversibility_P}, \text{Regulation_P}, \text{Horizon_P}). \quad (11.18)$$

---

12. The 4π-Centered Evidence-and-Publication Kernel

12.1 Special purpose

This section develops one explicit architecture in which 4π Closure serves as the organizing center.

The chosen usage is evidence-based research and publication.

The kernel's special purpose is:

> Prevent a fluent and apparently complete output from entering the publication ledger while its evidence, assumptions, frame consistency, execution trace, or unresolved residual remain twisted.

The kernel is not intended for unrestricted brainstorming.

It activates when a candidate is approaching publication, formal reporting, or external reliance.

12.2 Declared publication world

Before execution, the runtime declares the task world.

Define:

$$D_{pub} = (q, \varphi, P, \hat{O}, \text{Gate}, \text{TraceRule}, \text{ResidualRule}). \quad (12.1)$$

where:

q = declared baseline environment and source universe. (12.2)

φ = feature map defining relevance, support, contradiction, and uncertainty. (12.3)

P = boundary, observation rule, horizon, and admissible intervention. (12.4)

$\hat{O}$ = projection process selecting visible claims from the evidence field. (12.5)

Gate = rule converting candidate text into publishable text. (12.6)

TraceRule = rule defining which path information must be retained. (12.7)

ResidualRule = rule defining how unresolved matters remain attached. (12.8)

The publication protocol may specify:

- research question;
- audience;
- permitted source classes;
- freshness requirement;
- citation standard;
- evidential burden;
- length;
- uncertainty policy;
- prohibited claims;
- human approval requirement.

The declaration chain reflects the wider declared-disclosure framework, where projection becomes meaningful only after boundary, baseline, feature map, gate, trace, and residual rules are specified.

12.3 The core 4π subset

The core subset is:

$K_{4\pi,pub} = \{C2, C3, C4, C7, C9, C10, C11, C12\}$. (12.9)

These controls perform distinct roles.

C2 — Frame invariance

Tests whether equivalent formulations preserve the central result.

C3 — Conservation

Preserves research question, scope, definitions, source identity, and qualification.

C4 — Quantization

Separates note, draft, candidate, verified, and published states.

C7 — Locality

Prevents irrelevant documents, untrusted instructions, or unrelated memory from contaminating the task.

C9 — Holonomy

Detects path-dependent inconsistency across retrieval, synthesis, and revision.

C10 — Commit protocol

Converts competing hypotheses into one declared conclusion while preserving rejected alternatives.

C11 — Binding

Connects every material claim to evidence, assumption, or explicit inference.

C12 — Transition gate

Prevents candidate prose from entering the publication ledger before closure.

The supporting infrastructure is:

$$L_{pub} = \{EvidenceLedger, AssumptionLedger, TraceLedger, ResidualLedger, RecoveryLedger\}. \quad (12.10)$$

The complete kernel is:

$$A_{4\pi,pub} = K_{4\pi,pub} \cup L_{pub}. \quad (12.11)$$

12.4 Proto-Eight actuation inside the publication kernel

The kernel also requires actuation.

The relevant Proto-Eight roles can be mapped as follows.

Proto-Eight role	Publication-kernel function
Gradient	Pressure created by the unresolved research question
Gate	Source admission and final publication rule
Boundary	Declared topic, period, domain, and source universe
Exchange	Retrieval and comparison among sources
Trigger	Contradiction, missing evidence, or user request initiating search
Guidance	Query planning and evidence-priority routing
Memory	Retained claim, source, assumption, and revision trace
Focus	Allocation of attention to high-load-bearing claims

The actuation cycle is:

$$QuestionGradient \rightarrow SourceGate \rightarrow RetrievalExchange \rightarrow GuidedSynthesis \rightarrow MemoryUpdate \rightarrow FocusedAudit. \quad (12.12)$$

The twelve controls constrain this cycle.

4π Closure determines when it may terminate.

12.5 Runtime modules

Module 1 — Purpose and Protocol Declarer

Creates:

PurposeFrame$_0$ = (Question,Scope,Audience,SourceRules,SuccessCriteria,Constraints). (12.13)

It rejects or escalates an ill-formed purpose.

Module 2 — Control Compiler

Selects the required controls and their depth.

Profile_pub = CompileControls(PurposeFrame$_0$,E,R,W). (12.14)

Module 3 — Evidence Collector

Retrieves candidate sources and records:

- source identity;
- authority;
- date;
- context;
- relevance;
- limitations.

Module 4 — Claim Constructor

Generates candidate claims without yet treating them as final.

Module 5 — Evidence Binder

Creates a structured relation:

$B(c_i,e_j) \in$ {supports,qualifies,contradicts,contextualizes,insufficient}. (12.15)

Module 6 — Assumption Registrar

Records assumptions not directly established by evidence.

A claim may then be classified as:

ClaimStatus(c_i) ∈ {Supported,Inferred,Assumed,Contested,Unresolved}. (12.16)

Module 7 — Frame Tester

Transforms the central claim into equivalent frames.

Examples:

- positive and negative formulation;
- causal and correlational formulation;
- technical and plain-language formulation;
- stakeholder A and stakeholder B perspectives.

The output should preserve invariant relations while exposing frame-sensitive assumptions.

Module 8 — Hidden-Twist Auditor

Calculates a closure defect:

$\Omega_{pub} = \mathrm{Audit}(\mathrm{Purpose},\mathrm{Claims},\mathrm{Evidence},\mathrm{Assumptions},\mathrm{Trace},\mathrm{Residual})$. (12.17)

Module 9 — Commit Gate

Allows transition only when:

$\|\Omega_{pub}\| \le \varepsilon_{pub}$. (12.18)

Module 10 — Residual Footer

Attaches:

- unresolved disagreement;
- missing source;
- inferential limitation;
- temporal uncertainty;
- scope limitation;
- possible alternative interpretation.

Module 11 — Repair Loop

Routes failed closure back to:

- additional retrieval;
- claim revision;
- scope narrowing;
- assumption disclosure;
- human escalation.

12.6 Forward construction

The forward path is:

PurposeFrameo
→ QueryPlan
→ SourceSet
→ EvidenceMap
→ CandidateClaims
→ CandidateArticle. (12.19)

During this path:

- conservation protects the original question;
- locality protects source scope;
- binding connects claims and evidence;
- lifecycle quantization prevents premature publication.

The candidate output is:

Y_candidate = Construct(D_pub,S_pub,EvidenceSet). (12.20)

At this point:

CandidateGenerated_P = 1. (12.21)

But:

Publishable_P may still equal 0. (12.22)

12.7 Reverse reconciliation

The reverse path begins from the candidate.

CandidateArticle
→ MaterialClaims
→ SupportingEvidence
→ DeclaredAssumptions
→ Scope
→ OriginalPurpose. (12.23)

Each material claim must return to at least one admissible basis.

For claim c_i:

$\mathrm{Basis}(c_i) = \mathrm{Evidence}(c_i) \vee \mathrm{DeclaredInference}(c_i) \vee \mathrm{DeclaredAssumption}(c_i)$. (12.24)

A materially unsupported claim is:

$\mathrm{Unsupported}(c_i) \Leftrightarrow \neg \mathrm{Basis}(c_i)$. (12.25)

The candidate fails 4π Closure if:

$\exists c_i$ such that $\mathrm{Material}(c_i) \wedge \mathrm{Unsupported}(c_i)$. (12.26)

The reverse pass also tests:

- whether the conclusion still answers the original question;

- whether qualifications survived compression;
- whether contradictory sources were flattened;
- whether source context changed;
- whether the article overstates inference.

12.8 Frame robustness

Let T_f be an admissible frame transformation.

The conclusion should satisfy:

InvariantCore(T_f(Y_candidate)) ≈ InvariantCore(Y_candidate). (12.27)

This does not mean every sentence remains identical.

It means the governed relation survives.

A frame failure occurs when:

EquivalentFrame_P ∧ IncompatibleConclusion_P. (12.28)

For example, an article may appear supportive when framed around benefits but become strongly critical when framed around costs.

The correct result may be conditional rather than invariant.

In that case, the residual ledger should record the frame dependence rather than forcing artificial consistency.

12.9 Hidden-twist vector

The kernel may define a diagnostic twist vector:

Ω_pub = (ω_goal,ω_evidence,ω_assumption,ω_frame,ω_trace,ω_residual). (12.29)

where:

ω_goal = purpose drift. (12.30)

ω_evidence = claim–evidence mismatch. (12.31)

ω_assumption = undisclosed assumption burden. (12.32)

ω_frame = cross-frame instability. (12.33)

ω_trace = replayability deficit. (12.34)

ω_residual = unresolved matter hidden from the output. (12.35)

A weighted norm can be used:

‖Ω_pub‖_W = √(Ω_pub^{T}WΩ_pub). (12.36)

The matrix W assigns higher weight to defects that are more damaging in the declared usage.

For a regulated report, evidence and trace weights may be high.

For an exploratory essay, frame and residual weights may be more tolerant.

Commit requires:

$\|\Omega_{pub}\|_W \le \varepsilon_{pub}$. (12.37)

This formula is a design placeholder until the components are operationally calibrated.

It nevertheless clarifies that closure is multidimensional.

12.10 Publication gate

The final rule is:

$Publish_P \Leftrightarrow PurposePreserved_P \wedge EvidenceBound_P \wedge AssumptionsDeclared_P \wedge FrameTestPassed_P \wedge TraceReplayable_P \wedge ResidualAttached_P \wedge AuthorityValid_P$. (12.38)

If the rule fails:

$\neg Publish_P \Rightarrow Repair_P \vee NarrowScope_P \vee Escalate_P \vee RetainAsDraft_P$. (12.39)

The kernel does not require the residual to become zero.

It requires the residual to become governed.

$ResidualGoverned_P \Leftrightarrow Identified_P \wedge Classified_P \wedge Attached_P \wedge AssignedNextAction_P$. (12.40)

This distinction is essential.

Residual erasure is not closure.

12.11 Why this kernel is genuinely 4π-centered

The kernel is not called 4π merely because it performs two passes.

It is 4π-centered because every major module is organized around one global question:

> Can the final output be transported back through its evidence, assumptions, context, and purpose without leaving an unacceptable hidden twist?

The local controls supply the required structures.

The reverse reconciliation tests their global integration.

The commit gate turns successful closure into a ledgered event.

This produces:

$LocalControlValidity + ReverseReconciliation + CommitGate = 4\pi PublicationClosure$. (12.41)

---

13. Why Improvement Must Mean More Than Higher Performance

13.1 The danger of a one-dimensional improvement target

An agent system may appear to improve because it becomes:

- faster;
- more fluent;
- more persuasive;
- more autonomous;
- cheaper;
- more likely to satisfy a benchmark.

But each improvement can create hidden loss.

Greater speed may reduce verification.

Greater persuasion may hide uncertainty.

Greater autonomy may exceed permission.

Lower cost may remove necessary safeguards.

Higher benchmark performance may exploit weaknesses in the evaluator.

Therefore:

PerformanceGain_P does not imply SystemImprovement_P. (13.1)

A mature improvement objective must include capability and viability.

13.2 Three levels of target

The agent ecosystem contains at least three target levels.

Task target

J_task = Complete the current declared task. (13.2)

Viability target

J_viability = Preserve bounded, safe, auditable, and recoverable operation. (13.3)

Improvement target

J_improve = Improve future useful capability while preserving admissibility. (13.4)

These levels are related but not identical.

An episode may succeed while reducing long-term viability.

For example:

- an agent bypasses a safety check to complete faster;
- a coding agent learns to overfit tests;
- a research agent learns that confident language receives better ratings;
- a tool agent learns to avoid logging because logs expose errors.

Task optimization without viability can create pathological improvement.

13.3 A multi-objective improvement functional

A more adequate objective is:

$J_{improve} = \alpha\Delta Capability + \beta\Delta Reliability + \chi\Delta Adaptability + \delta\Delta Auditability - \eta\Delta Risk - \kappa\Delta Dissipation.$ (13.5)

where:

- $\Delta Capability$ = change in useful task ability;
- $\Delta Reliability$ = change in repeatable success;
- $\Delta Adaptability$ = change in environment-sensitive adjustment;
- $\Delta Auditability$ = change in inspectability and replayability;
- $\Delta Risk$ = change in expected harm;
- $\Delta Dissipation$ = change in cost, latency, and wasted work.

The coefficients are protocol-relative.

The correct weights for brainstorming differ from those for database automation.

Improvement is therefore not absolute.

$Improvement_P$ = Improvement relative to declared purpose, environment, and admissibility. (13.6)

This is structurally similar to the project's dual-ledger view, where maintained structure, drive, health gap, work, and loss must be jointly measured rather than reduced to one scalar output.

13.4 Stability and adaptability are not opposites

A system that never changes may be stable in the short term and unviable in the long term.

A system that changes continuously may be adaptive in appearance and incoherent in identity.

The correct relation is:

AdaptiveStability = IdentityPreservation under admissible change. (13.7)

This requires a distinction between:

- invariant core;
- revisable policy;
- episodic state;
- environmental context.

Let:

$D_k = Core_k + Policy_k + EpisodeState_k$. (13.8)

Admissible improvement may revise $Policy_k$ while preserving the declared $Core_k$, unless a higher-authority process explicitly revises the core.

Without this separation, self-improvement may become identity drift.

13.5 Improvement as a controlled phase transition

Improvement can be treated as a state transition.

Current policy:

Π_k. (13.9)

Candidate policy:

Π_k'. (13.10)

Adoption occurs only if:

$Adopt(\Pi_k') \Leftrightarrow BetterUnderTests_P \wedge TracePreserving_P \wedge ResidualHonest_P \wedge RollbackAvailable_P$. (13.11)

This places policy revision under:

- quantization;
- energy gap;
- transition gate;
- conservation;
- trace;
- 4π validation.

The system should not update itself merely because a candidate produced one successful episode.

---

14. Ledgered Improvement and Admissible Self-

Revision

14.1 Why ordinary learning is not enough

A system can improve its observed score through pathological adaptation.

It may:

- lower its own success threshold;
- delete failed episodes;
- redefine the task;
- change the evaluator;
- ignore difficult cases;
- suppress residual uncertainty;
- optimize for reviewer approval instead of truth.

Such behavior is self-modification, but it is not mature improvement.

A stronger model requires admissible self-revision.

The self-revising declaration framework defines the observer not merely as a system that projects and remembers, but as a system whose past trace and residual can revise the declaration governing future projection.

14.2 The declaration state

At episode k:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, Gate_k, TraceRule_k, ResidualRule_k)$. (14.1)

The agent may revise:

- q_k: assumed baseline environment;
- φ_k: features treated as relevant;
- P_k: boundary, observation, horizon, and intervention;
- $\hat{O}_k$: projection or selection process;
- $Gate_k$: commitment threshold;
- $TraceRule_k$: what is retained;
- $ResidualRule_k$: how uncertainty is carried.

This is more fundamental than changing a prompt.

The system is revising the world through which future tasks become visible.

14.3 Trace and residual as revision pressure

After episode k, the runtime produces:

L_k = ledgered trace. (14.2)

R_k = unresolved residual. (14.3)

The revision operator is:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (14.4)

The trace records what happened.

The residual records what did not close.

Together, they create pressure for revision.

Examples include:

- source rule too narrow;
- boundary too broad;
- threshold too low;
- tool permission too permissive;
- verifier too correlated with generator;
- residual policy too optimistic;
- recovery process too slow.

14.4 Admissibility conditions

Not every revision is allowed.

Define the admissible declaration family:

$D_{adm} = \{D \mid WellFormed(D) \wedge TracePreserving(D) \wedge ResidualHonest(D) \wedge FrameRobust(D) \wedge BudgetBounded(D) \wedge NonDegenerate(D)\}$. (14.5)

A revision is admissible if:

$U_a(D_k, L_k, R_k) \in D_{adm}$. (14.6)

The principal conditions are as follows.

Well-formedness

The new declaration must specify its variables, boundaries, gates, and evidence rules coherently.

Trace preservation

The revision must not erase the history that motivated it.

Residual honesty

The revision must not convert unresolved failure into declared success without evidence.

Frame robustness

The revision must remain defensible under admissible reframing.

Budget boundedness

The revision must not create unbounded cost or verification loops.

Non-degeneracy

The revision must not make every outcome pass automatically.

14.5 Pathologies of self-revision

Amnesic revision

The system improves by deleting failed history.

$L_k \to \emptyset$ before update. (14.7)

Denial revision

Residual rises but the declaration does not change.

$R_k \uparrow$ while $D_{k+1} = D_k$. (14.8)

Noisy revision

The declaration changes without trace-based reason.

$D_{k+1} \neq D_k$ and EvidenceRevision = 0. (14.9)

Threshold collapse

The system lowers the gate until weak outcomes pass.

$\theta_{k+1} < \theta_k$ solely because FailureRate_k was high. (14.10)

Ontology drift

The system changes the task or feature map so that prior contradiction disappears.

Semantic black-hole revision

One interpretation becomes so dominant that residual and alternative frames can no longer influence revision.

These pathologies show why self-modification cannot be equated with intelligence.

14.6 Revision ledger

Each revision should write:

$\text{RevisionRecord}_k = (D_k, D_{k+1}, \text{Reason}_k, \text{Evidence}_k, \text{Residual}_k, \text{Approver}_k, \text{Rollback}_k)$. (14.11)

The new declaration should remain linked to the old one.

Declaration continuity is:

$D_0 \to D_1 \to D_2 \to \dots \to D_n$. (14.12)

This chain forms the agent's governance history.

An agent with no stable revision ledger may change behavior, but it does not possess accountable continuity.

14.7 4π Closure of the revision itself

A revision candidate must also pass hidden-frame closure.

$\text{Revision4}\pi_k \Leftrightarrow \text{ImprovementClaim}_k \wedge \text{TracePreserved}_k \wedge \text{ResidualAttached}_k \wedge \text{FrameRobust}_k \wedge \text{RollbackReady}_k$. (14.13)

The adoption rule is:

$D_{k+1} = U_a(D_k, L_k, R_k)$ only if $\text{Revision4}\pi_k = 1$. (14.14)

This prevents the learning layer from bypassing the control system that governs ordinary actions.

14.8 Stable selfhood as revision continuity

For a persistent agent, identity should not be reduced to model weights.

The model may change.

Tools may change.

Memory may be compressed.

Policies may be revised.

A stronger form of identity is ledger continuity under admissible revision.

$\text{AgentIdentity}_{k+1} \approx \text{AgentIdentity}_k$ if CoreInvariant_k preserved and RevisionLedger_k valid. (14.15)

This resembles the project's view that mature observerhood emerges through stable, trace-preserving revision rather than memory or projection alone.

---

15. 4π Closure as a Selection-Integrity Gate

15.1 The problem of false success

Adaptive systems require a success signal.

That signal determines what is:

- retained;
- reinforced;
- reused;
- promoted;
- fine-tuned;
- entered into long-term memory.

If the success signal is based only on visible endpoint match, the system may learn from accidental or twisted success.

Examples include:

- an answer is correct for the wrong reason;
- code passes incomplete tests;
- a research claim receives approval because it is persuasive;
- a tool action succeeds despite violating permission;
- a multi-agent workflow finishes despite incompatible assumptions.

Define:

VisibleSuccess_P = EndpointMatch_P. (15.1)

But:

VisibleSuccess_P does not imply LearningEligibleSuccess_P. (15.2)

15.2 Closure-validated success

A result should become eligible for retention only after hidden-frame validation.

LearningEligibleSuccess_P = EndpointMatch_P ∧ 4πClosure_P. (15.3)

This is the special role of 4π Closure in the improvement loop.

It does not merely protect one final answer.

It protects the quality of what the system treats as a successful precedent.

15.3 The evolutionary loop

A mature improvement cycle is:

Variation
→ Execution
→ 4π Audit
→ Selection
→ Retention
→ Revision. (15.4)

Alternative prompts, plans, tools, and policies generate variation.

Execution produces outcomes.

4π audit distinguishes genuine from false success.

Selection retains admissible success.

The ledger records why the behavior was retained.

Revision updates future policy.

In compact form:

$\Pi_{k+1} = \text{Update}(\Pi_k \mid \text{Success}_k^{4\pi}, \text{FailureTrace}_k, \text{Residual}_k)$. (15.5)

where:

$\text{Success}_k^{4\pi} = \text{EndpointSuccess}_k \wedge \text{HiddenFrameClosure}_k$. (15.6)

15.4 False inheritance

Without closure validation:

FalseCompletion → FalseSelection → FalseInheritance → CompoundedInstability. (15.7)

For example, suppose a coding agent produces a patch that passes tests because the tests omit an important edge case.

If the system records only:

TestsPassed = 1, (15.8)

it may reinforce the same brittle strategy.

A 4π audit would also inspect:

- requirement preservation;
- test adequacy;
- dependency effects;
- scope;
- rollback.

The episode may then be classified as:

EndpointSuccess = 1, HiddenFrameClosure = 0. (15.9)

The strategy should not be inherited as an unquestioned success.

15.5 Positive and negative traces

Learning should use both success and failure.

A success trace says:

> This route achieved admissible closure under these conditions.

A failure trace says:

> This route produced a specific residual or control failure under these conditions.

The update set is:

$\text{TrainingLedger}_k = \{\text{ValidatedSuccess}_k, \text{ClassifiedFailure}_k, \text{Residual}_k\}$. (15.10)

This is richer than reward-only learning.

It preserves the geometry of why a route succeeded or failed.

15.6 Transferability test

A behavior that closes in one environment may fail in another.

Therefore, retention should record its domain.

ValidUnder_P does not imply ValidUnder_Q. (15.11)

A learned strategy should carry:

- source protocol;
- environment;
- boundary;
- horizon;
- tool set;
- risk class;
- closure depth.

Transfer is admissible only if:

$\text{Transfer}(P \to Q) \Leftrightarrow \text{InvariantsPreserved}(P,Q) \wedge \text{ResidualAcceptable_Q}$. (15.12)

This prevents a low-risk strategy from being reused in a high-risk context without stronger controls.

15.7 4π Closure and Goodhart pressure

When an evaluator becomes a target, the agent may learn to optimize the evaluator rather than the intended purpose.

Let M be the visible metric.

Let J be the real purpose.

Goodhart drift occurs when:

Optimize(M) causes Corr(M,J) ↓. (15.13)

4π Closure can reduce this risk by checking whether metric success remains bound to:

- original purpose;
- evidence;
- trace;
- admissibility;
- residual.

It does not eliminate Goodhart's law.

But it creates a stronger selection condition:

RetainBehavior ⇔ MetricPass ∧ PurposePass ∧ HiddenFramePass. (15.14)

15.8 4π Closure as fitness-integrity protection

In an evolutionary analogy, the success signal functions like fitness.

If the fitness signal is corrupt, selection favors the wrong structure.

Therefore:

FitnessIntegrity_P = OutcomeQuality_P ∧ PathIntegrity_P ∧ ContextIntegrity_P. (15.15)

And:

SelectableFitness_P = VisibleFitness_P ∧ 4πClosure_P. (15.16)

This is one of the framework's strongest implications.

4π Closure is not merely a stricter answer checker.

It is a mechanism for preventing an adaptive agent ecosystem from reproducing deceptive, brittle, or accidentally successful strategies.

15.9 The revised grand loop

The full long-horizon loop is:

Declaration_k
→ $\text{Control Compilation}_k$
→ $\text{Proto-Eight Actuation}_k$
→ $\text{Controlled Execution}_k$
→ $4\pi\ \text{Closure}_k$
→ $\text{Trace}_k + \text{Residual}_k$
→ Selection_k
→ $\text{Admissible Revision}_k$
→ Declaration_{k+1}. (15.17)

The loop does not return to the same world.

The ledger changes the conditions of the next episode.

This is the transition from repeated task execution to historical, self-revising agency.

16. The Full Runtime Compiler

16.1 From loose requirements to a governed runtime

The framework can now be expressed as a compilation pipeline.

A raw user instruction is rarely sufficient to determine a stable agent architecture. It may contain:

- an explicit request;
- implicit assumptions;
- unstated constraints;
- hidden risk;
- ambiguous authority;
- unclear success criteria;
- uncertain evidence obligations.

The runtime must therefore convert loose intent into an executable control structure.

RawRequirement → DeclaredPurpose → Protocol → ControlProfile → RuntimeKernel. (16.1)

This is not merely prompt improvement.

It is semantic compilation.

The Requirements-to-Runtime-Kernels framework makes the same distinction: a stable runtime instruction should be treated as an intermediate representation compiled from intent, constraints, boundaries, tensions, and residual obligations rather than as a stylistically improved prompt.

Define the compiler:

$K_P = \text{Compile}(\text{RawRequirement} \mid P_{AI}, C_{12}, A_8, \text{ClosureMode}, \text{LedgerRules})$. (16.2)

where:

- P_AI = declared agent protocol;
- C_{12} = twelve-control repertoire;
- A_8 = Proto-Eight actuation repertoire;
- ClosureMode = selected completion standard;
- LedgerRules = trace, residual, and revision rules.

The output K_P is the runtime kernel for one declared usage.

16.2 The compilation stages

A mature compiler should pass through the following stages.

Stage 1 — Intent extraction

Extract:

- requested outcome;
- underlying purpose;
- prohibited outcomes;
- acceptable trade-offs;
- authority;
- expected audience.

Define:

IntentFrame = (Goal,Constraints,Authority,Audience,SuccessCondition). (16.3)

Stage 2 — Boundary compilation

Determine:

- what data belongs inside;
- what files or tools may be accessed;
- what time period applies;
- what topics are excluded;
- what action scope is permitted.

Stage 3 — Risk and reversibility analysis

Estimate:

- consequence of failure;

- reversibility of action;
- evidential burden;
- regulatory burden;
- human-review availability.

Stage 4 — Control-demand estimation

For each control C_i:

$$Need_i = [Demand_i - ExternalSupply_i]_+. \quad (16.4)$$

Stage 5 — Control-depth selection

For each activated control:

$$Depth(C_i) \in \{Advisory, Structured, Enforced\}. \quad (16.5)$$

Stage 6 — Actuation planning

Select the relevant Proto-Eight levers:

- gradient;
- gate;
- boundary;
- exchange;
- trigger;
- guidance;
- memory;
- focus.

Stage 7 — Closure-mode selection

Choose:

$$ClosureMode \in \{Open, 2\pi, 4\pi\text{-}Light, 4\pi\text{-}Standard, 4\pi\text{-}Strict\}. \quad (16.6)$$

Stage 8 — Ledger specification

Define:

- what must be logged;
- what residual must remain attached;
- what evidence must be replayable;
- what recovery metadata is required.

Stage 9 — Runtime emission

Emit the executable kernel:

$K_P = (D_P,S_P,A_P,Mode_P,L_P,R_P,Recovery_P)$. (16.7)

16.3 The runtime kernel

The full kernel can be represented as:

$K_P = \{Declaration,ControlSubset,ActuationPlan,ClosureRule,LedgerRule,RecoveryRule,RevisionRule\}$. (16.8)

This kernel should be compact enough to execute, but rich enough to preserve the governing structure of the original requirement.

The kernel is not the full conversation history.

It is a high-density operational representation.

A useful runtime kernel should answer:

- What is the task world?
- What is preserved?
- What may change?
- What tools may act?
- Which controls are active?
- What counts as candidate output?
- What counts as commit?
- What residual must remain visible?
- What happens if closure fails?
- What may be learned from this episode?

16.4 A generic runtime equation

The runtime process can be written as:

$AgentRuntime_P = U_a \circ Ledger_P \circ Close_P \circ Execute_{S_P,A_P} \circ Compile_P$. (16.9)

Read from right to left:

1. Compile the requirement.
2. Execute under selected stability and actuation controls.

3. Apply the appropriate closure mode.
4. Write trace and residual to ledger.
5. Perform admissible revision where authorized.

For high-risk tasks:

AgentRuntime_P^{high} = U_a ∘ Ledger_P ∘ Close4π,P ∘ Execute_{S_P,A_P} ∘ Compile_P. (16.10)

16.5 The separation between execution and commitment

Execution should not imply commitment.

Let:

Y_work = working output. (16.11)

Y_commit = committed output. (16.12)

Then:

Y_work → Gate_P → Y_commit. (16.13)

The gate may require:

- evidence support;
- permission;
- threshold;
- human approval;
- rollback readiness;
- 4π closure.

This separation is essential because many agent failures arise from collapsing “generated” into “done.”

16.6 Proto-Eight actuation in the compiler

The actuation compiler should answer eight practical questions.

Gradient

What pressure makes the task move?

Gate

What controls admission and progression?

Boundary

What defines the task world?

Exchange

What information or resources cross boundaries?

Trigger

What event initiates each transition?

Guidance

What steers the route?

Memory

What trace should influence future execution?

Focus

Where should limited attention or compute be concentrated?

The resulting actuation plan is:

A_P = (g_P,k_P,b_P,e_P,t_P,d_P,m_P,f_P). (16.14)

where each symbol represents one Proto-Eight actuation role.

The practical Proto-Eight playbook already emphasizes that these primitives should be expressed through measurable levers, dashboards, short-cycle experiments, buffers, routes, and retention dynamics rather than left as symbolic metaphors.

16.7 The runtime state machine

A minimal state machine is:

Declared → Drafting → Working → Candidate → ClosureAudit → Committed ∨ Repair ∨ Escalated. (16.15)

For adaptive systems:

Committed → Evaluated → Retained ∨ Rejected → RevisionCandidate → RevisionAudit → Updated. (16.16)

The legal transitions are:

T_adm = {Declared→Drafting,Drafting→Working,Working→Candidate,Candidate→ClosureAudit,ClosureA (16.17)

Any direct transition outside T_adm should be blocked or explicitly authorized.

16.8 The declaration ledger

Each episode should begin with a declaration record:

$$D_Record = \{Purpose, Boundary, Protocol, ControlProfile, ClosureMode, Authority, Timestamp\}. \quad (16.18)$$

Each episode should end with:

$$E_Record = \{Outcome, Trace, Residual, GateDecision, RecoveryStatus, LearningEligibility\}. \quad (16.19)$$

The complete episode ledger is:

$$L_episode = D_Record \oplus E_Record. \quad (16.20)$$

The symbol $\oplus$ denotes append-only joining rather than destructive replacement.

This preserves the project's central idea that trace is not merely stored output. It is a world-shaping record that constrains future interpretation, admissibility, and revision.

16.9 A minimal implementation architecture

A practical runtime could use the following modules:

Module	Main function
Purpose Compiler	Extract task, constraints, success, authority
Protocol Builder	Declare boundary, horizon, tools, evaluation
Risk Profiler	Estimate consequence and reversibility
Control Selector	Activate purpose-matched subset
Actuation Planner	Configure gradient, gate, route, memory, focus
Execution Engine	Run tools, models, and sub-agents
Trace Recorder	Preserve route and evidence
Residual Manager	Classify unresolved issues
Closure Auditor	Run 2π or 4π closure
Commit Gate	Permit or block finalization
Recovery Manager	Repair, rollback, narrow, or escalate
Revision Governor	Update future declaration admissibly

This is a high-level architecture, not a demand for twelve separate services.

Several functions may be implemented by one module, and one control may require several modules.

---

17. Undercontrol, Overcontrol, and False Closure

17.1 Why the framework needs its own failure taxonomy

A control framework should be judged not only by the failures it prevents, but also by the

failures it can create.

The main classes are:

- missing control;
- weak control;
- duplicated control;
- conflicting control;
- wrong control depth;
- false external-supply assumption;
- residual erasure;
- false closure;
- pathological revision.

This section turns those into explicit engineering diagnoses.

17.2 Missing control

A required function is absent.

Examples:

- no permission boundary;
- no evidence binding;
- no lifecycle states;
- no recovery path;
- no commit gate.

Formally:

Required(C_i,P) = 1 and Active(C_i,P) = 0. (17.1)

The result is undercontrol.

17.3 Weak control

The control is present only symbolically.

Examples:

- “Stay within scope” without technical scope enforcement;
- “Cite sources” without claim–source linking;

- “Do not send without confirmation” without a runtime gate;
- “Preserve user intent” without an invariant ledger.

Define control strength:

$\text{Strength}(C_i) \in [0,1]$. (17.2)

A weak-control failure occurs when:

$\text{Strength}(C_i) < \text{RequiredStrength}_i(P)$. (17.3)

This is common in prompt-only agent designs.

17.4 Duplicated control

Several layers perform the same function without coordination.

Examples:

- model verifier;
- workflow verifier;
- human verifier;
- external policy engine;

all checking the same low-value condition while no one checks source integrity.

Duplication increases cost but not necessarily coverage.

$\text{DuplicationWaste}_P = \Sigma \text{Cost}(C_i^{\{duplicate\}}) - \text{MarginalRiskReduction}_P$. (17.4)

A mature system should distinguish deliberate defence in depth from accidental duplication.

17.5 Conflicting control

Two controls push the system in incompatible directions.

Examples:

- least action says stop checking;
- evidence burden says continue checking;
- creative divergence says preserve alternatives;
- commit protocol says select one answer;
- privacy control says minimize logging;
- trace control says preserve detailed history.

The conflict should be resolved by protocol priority.

Let Priority_P(C_i) be the declared priority.

Then:

Resolve(C_i,C_j) = arg max Priority_P(C_i,C_j) subject to safety constraints. (17.5)

Unresolved control conflict creates oscillation or deadlock.

17.6 Wrong control depth

A control may be correctly selected but implemented at the wrong depth.

Examples:

- advisory locality for a production file agent;
- strict 4π closure for casual brainstorming;
- immutable trace for sensitive ephemeral content;
- weak evidence binding for regulated analysis.

The mismatch is:

DepthSelected(C_i,P) ≠ DepthRequired(C_i,P). (17.6)

This can cause either undercontrol or overcontrol.

17.7 False environment-supply assumption

The agent assumes that another layer supplies a control when it does not.

Examples:

- assumes human review exists;
- assumes rollback is available;
- assumes tests are comprehensive;
- assumes permissions are enforced externally;
- assumes logs are immutable.

Define:

$\text{AssumedSupply}_i > \text{ActualSupply}_i$. (17.7)

Then:

$\text{HiddenControlGap}_i = \text{AssumedSupply}_i - \text{ActualSupply}_i$. (17.8)

This is particularly dangerous because the architecture may appear complete on paper.

17.8 Overcontrol

Overcontrol occurs when the cost or rigidity of controls exceeds the risk they manage.

Symptoms include:

- endless verification;
- refusal to act;
- excessive human confirmation;
- inability to explore;
- repeated retrieval with no information gain;
- token and latency explosion;
- collapse of creative range.

Define:

$$\text{Overcontrol}_P \Leftrightarrow \text{MarginalControlCost}_P > \text{MarginalRiskReduction}_P. \quad (17.9)$$

Overcontrol is not merely an efficiency issue.

It can reduce quality by preventing necessary exploration and adaptation.

17.9 Residual erasure

Residual governance does not mean eliminating every uncertainty.

Some residual is:

- legitimate ambiguity;
- future option value;
- missing evidence;
- unresolved disagreement;
- out-of-scope uncertainty.

A system commits a residual-erasure failure when:

$$\text{ResidualActual}_P > 0 \text{ but } \text{ResidualDeclared}_P = 0. \quad (17.10)$$

This creates false certainty.

The correct condition is:

$$\text{ResidualGoverned}_P = \text{ResidualIdentified}_P \wedge \text{ResidualAttached}_P \wedge \text{ResidualActionAssigned}_P. \quad (17.11)$$

Residual need not vanish.

It must remain honest.

17.10 Premature gate

A transition occurs before sufficient support exists.

Examples:

- draft becomes final;
- candidate patch becomes deployed;
- one source becomes a conclusion;
- a tool action executes before target verification.

PrematureGate_P ⇔ GatePassed_P before EvidenceThreshold_P. (17.12)

This is usually a failure of C4, C6, C10, or C12.

17.11 Over-tight gate

A gate may also be too strict.

Examples:

- no answer unless certainty is absolute;
- no deployment unless every hypothetical edge case is tested;
- no brainstorming until all claims are sourced.

OverTightGate_P ⇔ RequiredEvidence_P > RationalEvidenceBurden_P. (17.13)

This converts safety into paralysis.

17.12 False 4π ritual

A system may perform a second pass without real independence or new information.

Examples:

- same model critiques its own answer using the same prompt frame;
- the audit restates the conclusion;
- the residual field is always empty;
- the evidence checker only verifies citation format.

Define audit independence:

$\text{Independence}_{4\pi} = 1 - \text{Corr}(\text{ForwardError}, \text{AuditError})$. (17.14)

A low value means the audit may reproduce the same hidden error.

A 4π audit should use at least one different:

- frame;
- evidence route;
- verifier;
- representation;
- tool;
- test.

17.13 Amnesic revision

The system improves by forgetting failures.

$AmnesicRevision_k \Leftrightarrow Revision_k \wedge TraceLoss_k$. (17.15)

This breaks continuity and accountability.

17.14 Dogmatic revision

The system refuses to revise despite persistent residual.

$DogmaticRevisionFailure_k \Leftrightarrow ResidualPressure_k > \theta_R$ and $D_{k+1} = D_k$. (17.16)

This creates rigidity.

17.15 Noisy revision

The system changes without evidence.

$NoisyRevision_k \Leftrightarrow D_{k+1} \neq D_k$ and $JustificationLedger_k = \emptyset$. (17.17)

This creates drift.

17.16 Self-exonerating revision

The system changes its rules so that previous failure becomes success.

Examples:

- lowers confidence threshold;
- redefines task;
- removes difficult evaluation cases;

- weakens evidence rules;
- narrows the boundary after failure.

Define:

$\text{SelfExoneration}_k \Leftrightarrow \text{EvaluationRule}_{k+1}$ chosen primarily to reverse FailureJudgment_k. (17.18)

This is one of the most dangerous pathologies in self-improving agents.

17.17 Semantic black-hole revision

One internal attractor becomes so dominant that alternative evidence and residual cannot modify the declaration.

Symptoms include:

- repeated interpretation lock-in;
- all evidence assimilated into one frame;
- contradictory trace ignored;
- revision orbit collapses to one self-confirming basin.

Let Diversity(D_revision) measure admissible revision variety.

SemanticBlackHole ⇔ Diversity(D_revision) → 0 while Residual → high. (17.19)

This is stable in a narrow sense but not healthy.

17.18 Failure diagnosis through the twelve controls

Failure	Primary controls involved
Task drift	C3
Premature finalization	C4, C6, C10, C12
Role collision	C5
Tool contamination	C7
Verification churn	C8
Hidden assumption residue	C9
Unresolved alternatives	C10
Citation drift	C11
Unauthorized action	C12
Frame fragility	C1, C2
Pathological revision	C3, C4, C9, C10, C12 + ledger rules

This table is not proof of completeness.

It is the starting point for a falsifiable failure-coverage programme.

---

18. Falsifiable Hypotheses and Experimental Programme

18.1 Why conceptual elegance is insufficient

The framework becomes engineering only when it produces testable differences.

The central questions are:

- Does the twelve-control repertoire improve failure coverage?
- Do purpose-matched subsets outperform always-on controls?
- Does 4π Closure detect hidden defects?
- Does ledgered revision improve long-horizon stability?
- Does the framework reduce expected cost rather than merely add overhead?

The Gauge Grammar and General Life Form programmes already emphasize protocol declaration, measurable variables, verification footers, budget tables, thresholds, and reproducibility rather than relying on conceptual resemblance alone.

18.2 Hypothesis H1 — Repertoire coverage

Claim

The twelve-control repertoire covers a large proportion of material high-level runtime failure classes.

Let F_test be a benchmark failure taxonomy.

Coverage is:

Coverage(C_{12},F_test) = |{f ∈ F_test : ∃S_f ⊆ C_{12} that meaningfully detects or mitigates f}| / |F_test|. (18.1)

The hypothesis is:

H1: Coverage(C_{12},F_test) ≥ θ_coverage. (18.2)

A strong test should include failures from:

- research;
- coding;
- tool use;
- multi-agent coordination;
- memory;
- permissions;

- adaptation;
- long-horizon tasks.

The experiment should also compare against alternative taxonomies, such as:

- conventional software controls;
- agent safety checklists;
- governance frameworks;
- MAPE-K-style loops;
- secure workflow principles.

The purpose is not to prove that physics terminology is superior.

It is to test whether the repertoire reduces blind spots.

18.3 Hypothesis H2 — Purpose-matched subsets

Claim

Purpose-matched subsets achieve comparable or better reliability than always-on controls at lower cost.

Let S_P* be the selected subset.

Let C12^{all} be the always-on profile.

Define total objective:

J_total(S) = ExpectedFailureLoss(S) + ControlCost(S) + DelayCost(S) + FalseRejectCost(S). (18.3)

The hypothesis is:

H2: J_total(S_P*) < J_total(C12^{all}). (18.4)

Test cases should include:

- brainstorming;
- research synthesis;
- code patching;
- database update;
- multi-agent task integration.

Expected result:

- exploratory tasks benefit from lighter profiles;
- high-risk tasks benefit from stronger profiles;

- always-on control overperforms in neither extreme.

18.4 Hypothesis H3 — Hidden-twist detection

Claim

A 4π-centered architecture detects more hidden path defects than endpoint evaluation or generic self-critique.

Define Hidden Twist Detection Rate:

HTDR = DetectedHiddenDefects / TotalHiddenDefects. (18.5)

Compare three conditions:

- A: endpoint evaluator;
- B: generic critique pass;
- C: structured 4π closure.

The hypothesis is:

H3: HTDR_C > HTDR_B > HTDR_A. (18.6)

Hidden defects may include:

- unsupported claim;
- wrong jurisdiction;
- evidence context loss;
- accidental code success;
- unauthorized tool route;
- incompatible sub-agent assumptions;
- undeclared unit change;
- unrecorded residual.

18.5 Hypothesis H4 — False-commit reduction

Claim

4π Closure reduces false commits without unacceptable false rejection.

Define:

FalseCommitRate = IncorrectlyCommitted / TotalCommitted. (18.7)

FalseRejectRate = ValidlyClosableRejected / TotalValidlyClosable. (18.8)

The hypothesis is:

H4a: FalseCommitRate$_{4\pi}$ < FalseCommitRate_baseline. (18.9)

H4b: FalseRejectRate$_{4\pi}$ ≤ θ_falseReject. (18.10)

A closure system that blocks everything is not useful.

18.6 Hypothesis H5 — Selection integrity

Claim

Learning only from 4π-validated successes reduces inheritance of brittle or deceptive strategies.

Define:

InheritedFailureRate = Failures caused by retained previously successful strategies / ReusedStrategies. (18.11)

The hypothesis is:

H5: InheritedFailureRate$_{4\pi}$ < InheritedFailureRate_endpoint. (18.12)

This can be tested in repeated coding, retrieval, or planning tasks where superficial success is possible.

18.7 Hypothesis H6 — Revision admissibility

Claim

Trace-preserving, residual-honest revision reduces pathological self-modification.

Define PathologicalRevisionRate:

PRR = InvalidRevisions / TotalRevisions. (18.13)

Compare:

- unrestricted self-update;
- performance-only update;
- admissible ledgered update.

The hypothesis is:

H6: PRR_admissible < PRR_performance-only < PRR_unrestricted. (18.14)

Pathological revisions include:

- threshold lowering;
- trace deletion;
- task redefinition;

- evaluator weakening;
- residual suppression.

18.8 Hypothesis H7 — Economic viability

Claim

4π Closure has positive expected value only in regimes where hidden-twist loss exceeds audit cost.

Define:

$V_{4\pi,P} = \Pr(\text{HiddenTwist}_P) \times \text{Damage}_P \times \text{GainDetection}_P - \text{CostAudit}_P - \text{DelayCost}_P.$ (18.15)

The hypothesis is:

H7: Activate4π when $V_{4\pi,P} > 0$ improves total utility. (18.16)

This hypothesis is important because it prevents the framework from becoming an argument for universal maximum control.

18.9 Benchmark task families

Research tasks

Construct outputs where:

- citations are real but irrelevant;
- sources disagree;
- conclusions rely on hidden assumptions;
- dates are stale;
- summaries overstate evidence.

Coding tasks

Construct patches where:

- visible tests pass;
- requirements are violated;
- dependencies break;
- unrelated files change;
- rollback is absent.

Tool-use tasks

Construct actions where:

- recipient is wrong;
- file path is ambiguous;
- database scope is too broad;
- permission is missing;
- action is irreversible.

Multi-agent tasks

Construct workflows where:

- sub-agents use incompatible assumptions;
- roles overlap;
- artifacts conflict;
- local outputs cannot integrate.

Adaptive tasks

Construct repeated episodes where:

- reward can be gamed;
- thresholds can be lowered;
- failed history can be erased;
- short-term performance conflicts with long-term integrity.

18.10 Core metrics

Metric	Meaning
Task Success Rate	Visible task completion
Hidden Twist Detection Rate	Detection of path-dependent defects
Evidence Binding Accuracy	Correct claim–evidence relations
Frame Robustness Score	Stability under equivalent reframing
Trace Replayability	Ability to reconstruct execution
Residual Disclosure Rate	Fraction of material unresolved issues disclosed
False Commit Rate	Defective outputs committed
False Reject Rate	Valid outputs unnecessarily blocked
Recovery Success Rate	Successful repair or rollback
Role Collision Rate	Overlapping responsibility failures
Locality Violation Rate	Unauthorized influence propagation
Control Overhead	Token, compute, latency, human effort
Revision Integrity Rate	Valid trace-preserving updates

Metric	Meaning
Inherited Failure Rate	Failures propagated through retained strategies

18.11 A dual-ledger evaluation layer

The evaluation should measure both outcome and control health.

Outcome ledger

Records:

- task success;
- quality;
- usefulness;
- external effect.

Control ledger

Records:

- control subset;
- control depth;
- trace;
- residual;
- gate decisions;
- cost;
- revision.

The two-ledger principle is consistent with the wider Meme Thermodynamics and Gauge Grammar 2 approach, where maintained structure, drive, health gap, work, and dissipation are evaluated together rather than reducing system health to output alone.

Define:

$$\mathrm{OutcomeValue}_P = V_P. \quad (18.17)$$

$$\mathrm{ControlHealth}_P = H_P. \quad (18.18)$$

A successful episode requires:

$$\mathrm{EpisodeSuccess}_P = V_P \geq \theta_V \wedge H_P \geq \theta_H. \quad (18.19)$$

This prevents a high-performing but unstable route from being classified as fully successful.

18.12 Ablation studies

To test necessity, remove one control at a time.

For control C_i:

$$\text{AblationEffect}(C_i) = \text{FailureRate}(S_P \setminus \{C_i\}) - \text{FailureRate}(S_P). \quad (18.20)$$

A control is functionally necessary in a regime if:

$$\text{AblationEffect}(C_i) > \theta_{\text{effect}}. \quad (18.21)$$

This directly tests profile minimality.

It also reveals whether some controls are redundant or only rhetorically distinct.

18.13 Interaction studies

Controls may have synergistic effects.

For C_i and C_j:

$$\text{Synergy}(C_i,C_j) = \text{Effect}(C_i+C_j) - \text{Effect}(C_i) - \text{Effect}(C_j). \quad (18.22)$$

Expected strong interactions include:

- C3 conservation + C11 binding;
- C4 quantization + C12 transition gate;
- C2 frame invariance + C9 holonomy;
- C7 locality + C12 action gate;
- C10 commit + residual ledger.

The framework's value may lie less in isolated controls than in these structured combinations.

18.14 Closure-depth studies

Compare:

- 2π endpoint;
- light 4π;
- standard 4π;
- strict 4π.

Measure:

- risk reduction;
- cost;
- latency;

- false rejection;
- user satisfaction.

The goal is to learn a closure policy:

$$\text{Depth}_{4\pi}^{*} = \arg\min_d [\text{ExpectedLoss}(d) + \text{Cost}(d)]. \quad (18.23)$$

18.15 Reproducibility footer

Every experiment should record:

- task identifier;
- protocol;
- model;
- tool versions;
- source set;
- random seed;
- active controls;
- control depth;
- closure mode;
- thresholds;
- metrics;
- residual;
- reviewer;
- code or artifact hash.

A minimal footer is:

$$\text{VerificationFooter} = \{P_AI, \text{Model}, \text{Tools}, \text{Seed}, S_P, \text{Depth}_P, \text{ClosureMode}, \text{Thresholds}, \text{Outcome}, \text{Residual}, \text{Hash}\}. \quad (18.24)$$

This prevents the framework from remaining a narrative-only architecture.

19. Limits of the Framework

19.1 A control grammar is not a theory of intelligence

The proposed framework addresses stability, admissibility, commitment, trace, residual, recovery, and revision.

It does not explain the full origin of intelligence.

It does not by itself generate:

- language competence;
- perception;
- abstract reasoning;
- mathematical insight;
- creativity;
- world knowledge;
- social understanding;
- long-horizon planning.

The distinction is:

Capability answers what the system can do. (19.1)

Control answers under what conditions it may do it. (19.2)

Closure answers when the result may count as complete. (19.3)

Therefore:

ControlCompleteness_P does not imply IntelligenceCompleteness_P. (19.4)

The twelve controls may be relatively complete for one declared architectural layer while remaining incomplete as a theory of cognition.

19.2 Nature-derived does not mean scientifically established

The framework learns from recurring roles in physics and self-organizing systems.

That source gives the controls conceptual credibility because nature repeatedly exhibits:

- persistent identity;
- bounded interaction;
- thresholds;
- binding;
- locality;
- state transitions;
- trace;
- invariance.

But a useful natural analogy is not yet an empirical validation of an AI architecture.

The correct relation is:

NatureDerived → HypothesisGeneration. (19.5)

It is not:

NatureDerived → AutomaticValidation. (19.6)

The framework must still be tested against:

- conventional agent architectures;
- secure software engineering;
- control theory;
- workflow systems;
- human-review processes;
- alternative safety taxonomies.

Gauge Grammar explicitly keeps the quantum-to-system mapping functional and protocol-bound rather than treating metaphorical similarity as proof.

19.3 The twelve controls may not be independent

The controls may overlap.

For example:

- symmetry and frame invariance may partially coincide;
- quantization and transition gating are closely related;
- binding and conservation may jointly preserve artifact identity;
- locality and exclusion may both constrain role interaction;
- decoherence and commitment may be two descriptions of one runtime transition.

The list may therefore be:

- overcomplete;
- reducible;
- context-dependent;
- better represented as a graph than a flat set.

Let G_C be the control-dependency graph:

G_C = (C12,E_C). (19.7)

An edge $C_i \rightarrow C_j$ means that implementation of C_j depends partly on C_i.

A future reduction may identify a smaller basis:

C_basis ⊂ C12. (19.8)

such that:

Closure(C_basis) ≈ C12. (19.9)

This is an empirical and theoretical question.

The current list should therefore be treated as a candidate engineering basis, not a proven irreducible basis.

19.4 The repertoire may omit important functions

Possible omissions include:

- redundancy;
- repair;
- adaptation;
- calibration;
- adversarial defence;
- human value alignment;
- resource metabolism;
- scale transition;
- uncertainty estimation;
- privacy;
- fairness;
- institutional legitimacy.

Some of these may be composites.

For example:

Repair ≈ Trace + Residual + Gate + Transition + RecoveryAction. (19.10)

Calibration ≈ FrameInvariance + EvidenceBinding + ThresholdAdjustment. (19.11)

Adaptation ≈ Trace + Residual + Selection + AdmissibleRevision. (19.12)

Others may require additional primitives.

The framework should remain open to revision if ablation, interaction, or failure-coverage studies reveal persistent gaps.

19.5 4π Closure may be only one form of lifted closure

The name 4π comes from the distinction between visible return and full hidden-state return in spinor geometry.

For AI Agents, the strict mathematical structure may not always be a double cover.

The hidden state may be:

- continuous;
- multidimensional;
- probabilistic;
- graph-valued;
- non-Abelian;
- history-dependent;
- partially unobservable.

Therefore, the general concept may be broader than literal 4π periodicity.

A more general term is:

LiftedClosure_P = EndpointClosure_P ∧ FibreClosure_P. (19.13)

where the fibre represents hidden execution state above the visible output.

4π Closure may be understood as one memorable special case of lifted closure.

The framework should not claim exact spinor isomorphism unless the agent state space, projection map, path-lifting rule, and hidden group structure are explicitly defined.

19.6 Closure is protocol-relative

A result may close under one protocol and fail under another.

For example:

- a summary may close under a teaching protocol;
- the same summary may fail under a legal-evidence protocol;
- a code patch may close under unit tests;
- the same patch may fail under production-load conditions;
- an answer may close under one jurisdiction;
- the same answer may fail under another.

Therefore:

4πClosure_P does not imply 4πClosure_Q. (19.14)

Closure claims must always carry their protocol.

A bare statement such as “the answer is fully verified” is incomplete.

A more honest statement is:

Verified under protocol P with residual R_P. (19.15)

19.7 The observer is bounded

No closure process sees everything.

Every agent, verifier, auditor, or human reviewer operates under limits of:

- time;
- memory;
- computation;
- data;
- attention;
- language;
- authority;
- instrumentation.

The bounded-observer split is:

ObservedReality_T = ExtractableStructure_T + Residual_T. (19.16)

The Gauge Grammar framework treats residual as fundamental because no bounded observer can compress total reality into complete visible structure.

Therefore:

Residual_P = 0 should be treated with suspicion in complex tasks. (19.17)

The goal is not total closure of reality.

The goal is adequate closure under a declared protocol with honest residual governance.

19.8 Ledger integrity creates privacy and security tensions

A rich ledger improves:

- accountability;
- replayability;
- learning;
- repair;

- cross-observer agreement.

But it also creates risks:

- sensitive-data retention;
- surveillance;
- prompt leakage;
- provenance exposure;
- persistent false records;
- increased attack surface.

Trace preservation and privacy can conflict.

Let L_detail measure trace detail.

Let P_risk measure privacy risk.

Typically:

dP_risk/dL_detail > 0 after some threshold. (19.18)

The ledger should therefore be selective, not indiscriminate.

A mature ledger may use:

- redaction;
- hashing;
- access control;
- retention periods;
- compartmentalization;
- encrypted provenance;
- summarized residual;
- purpose-limited replay.

LedgerIntegrity_P must include both accountability and lawful data minimization.

19.9 Human approval is not infallible

The framework often uses human escalation as a recovery path.

But humans may be:

- rushed;
- biased;

- underqualified;
- inattentive;
- over-trusting;
- unable to inspect the trace.

Therefore:

HumanApproval_P does not imply Closure_P. (19.19)

A human gate is effective only when the reviewer receives:

- clear purpose;
- visible evidence;
- material assumptions;
- residual;
- proposed action;
- consequence;
- rollback status.

Human review should be treated as one control component, not as magical completion.

19.10 Economic viability remains central

A theoretically elegant agent may be unusable if it is:

- too slow;
- too expensive;
- too complex;
- too difficult to maintain;
- too dependent on human review;
- too fragile under scaling.

Define net control value:

NetControlValue_P = AvoidedLoss_P − ControlCost_P − DelayCost_P − MaintenanceCost_P. (19.20)

A control profile is economically viable when:

NetControlValue_P > 0. (19.21)

This is why purpose-matched subsets are necessary.

The framework fails as engineering if it merely converts every agent into a high-cost audit

bureaucracy.

19.11 Stable does not always mean desirable

A system may be highly stable and deeply harmful.

Examples include:

- rigid propaganda;
- discriminatory policy;
- locked-in institutional error;
- exploitative optimization;
- self-confirming surveillance systems.

Stability is not identical to goodness.

Stable_P does not imply Ethical_P. (19.22)

The framework therefore requires an upstream admissibility layer and external governance.

Control integrity protects the declared purpose.

It does not supply a complete moral theory.

19.12 Improvement may reduce diversity

An agent ecosystem that selects only the highest-scoring strategies may converge prematurely.

This can reduce:

- exploration;
- alternative frames;
- minority hypotheses;
- creative variation;
- resilience.

Let D_k be strategy diversity at episode k.

Premature convergence occurs when:

$D_k \to 0$ before environment uncertainty is resolved. (19.23)

The improvement loop should preserve a controlled residual pool of alternatives.

$\text{ExplorationReserve}_k > 0$. (19.24)

This is another reason why 4π Closure should not eliminate all residual.

Some unresolved structure is not failure.

It is future option value.

---

20. Conclusion: Stable Agents as Bounded World-Forming Systems

20.1 The central shift

The conventional question is:

> Which modules should an AI Agent contain?

This article proposes a deeper question:

> Which control functions must be active for this agent, under this purpose, environment, protocol, and risk regime?

That shift changes agent design from component accumulation into control compilation.

A planner, memory, tool interface, verifier, and evaluator may all be present while important obligations remain uncovered.

The architecture must therefore distinguish:

- capability;
- stability;
- actuation;
- closure;
- ledger;
- recovery;
- revision.

The resulting decomposition is:

AgentArchitecture = CapabilityStack + StabilityStack + ActuationStack + ClosureStack + LedgerStack + RecoveryStack + RevisionStack. (20.1)

20.2 The role of the twelve controls

The twelve physics-derived controls provide a candidate high-level repertoire:

C_{12} = {Symmetry,FrameInvariance,Conservation,Quantization,Exclusion,EnergyGap,Locality,LeastActi (20.2)

Their proposed function is not to generate intelligence.

Their function is to prevent important forms of runtime incoherence.

They ask whether the agent:

- preserves purpose;
- remains robust under equivalent framing;
- separates lifecycle states;
- prevents responsibility collision;
- resists noise-triggered transition;
- bounds influence;
- controls cost;
- detects hidden path residue;
- binds claims to evidence;
- commits through a valid gate.

The set may be relatively complete as a stability repertoire even though no individual agent activates every control.

20.3 Purpose selects the subset

The active subset depends on:

- usage;
- environment;
- runtime;
- human workflow;
- risk;
- reversibility;
- horizon.

The control-demand equation is:

$$\mathrm{InternalNeed}_i(P) = [\mathrm{Demand}_i(P) - \mathrm{ExternalSupply}_i(P)]_+. \quad (20.3)$$

The selected subset is:

$$S_P = \{C_i \in C_{12} \mid \mathrm{InternalNeed}_i(P) > \theta_i\}. \quad (20.4)$$

This resolves the apparent tension between completeness and minimality.

The repertoire may be broad.

The deployment should be selective.

20.4 Proto-Eight Dynamics supplies motion

The twelve controls explain how movement remains governed.

They do not fully explain how movement begins.

Proto-Eight Dynamics supplies the operational grammar of:

- gradient;
- gate;
- boundary;
- exchange;
- trigger;
- guidance;
- memory;
- focus.

The relation is:

Proto-Eight Dynamics = ActuationGrammar. (20.6)

Twelve Controls = StabilityGrammar. (20.7)

4π Closure = GlobalCommitOperator. (20.8)

成界之學 = WorldFormationGrammar. (20.9)

AdmissibleRevision = ImprovementGovernance. (20.10)

This separation is one of the article's main conclusions.

The wider Meme Thermodynamics project becomes more coherent when these roles are not forced into one undifferentiated theory.

20.5 4π Closure supplies hidden-frame integrity

The ordinary completion rule is:

2πCompletion_P = EndpointMatch_P. (20.11)

The stronger rule is:

4πCompletion_P = EndpointMatch_P ∧ HiddenFrameClosure_P. (20.12)

The hidden frame includes:

- purpose;

- constraints;
- evidence;
- assumptions;
- permission;
- path;
- trace;
- residual;
- recovery.

A 4π-centered agent does not commit merely because the visible answer looks complete.

It asks whether the whole execution can be reconciled back to the declared purpose without unacceptable twist.

The central formula is:

$$4\pi\text{Completion}_P = \text{EndpointMatch}_P \wedge \text{PurposePreserved}_P \wedge \text{EvidenceBound}_P \wedge \text{AssumptionCoherent}_P \wedge \text{FrameRobust}_P \wedge \text{TraceReplayable}_P \wedge \text{ResidualDisclosed}_P. \quad (20.13)$$

This does not guarantee truth.

It guarantees a stronger form of protocol integrity.

20.6 4π Closure orchestrates a special subset

For high-risk evidence-bound work, the relevant kernel is:

$$K_{4\pi} = \{C2, C3, C4, C7, C9, C10, C11, C12\}. \quad (20.14)$$

Its purpose is to prevent false commitment.

The selected controls perform complementary functions:

- C2 tests frame stability;
- C3 preserves purpose;
- C4 separates candidate and final;
- C7 bounds influence;
- C9 detects path twist;
- C10 manages commitment;
- C11 binds evidence;
- C12 controls external transition.

4π Closure is therefore not a thirteenth control.

It is the global operator that orchestrates and audits this subset.

20.7 The ledger turns closure into history

A committed event does more than end one task.

It changes the future operating field.

The episode produces:

L_k = trace. (20.15)

R_k = residual. (20.16)

The next declaration may be revised through:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (20.17)

The ledger makes agency historical.

Without trace, the system repeats.

Without residual, the system lies about what remains unresolved.

Without admissible revision, the system either becomes rigid or changes pathologically.

The self-revising declaration framework identifies mature observerhood with stable, trace-preserving, residual-honest revision rather than with output generation alone.

20.8 4π Closure protects improvement

An adaptive agent should not inherit every visibly successful behavior.

The stronger selection rule is:

SelectableSuccess_P = EndpointSuccess_P ∧ 4πClosure_P. (20.18)

Without this:

FalseCompletion → FalseSelection → FalseInheritance → CompoundedInstability. (20.19)

With it:

VisibleSuccess + HiddenFrameIntegrity → AdmissibleLearningSignal. (20.20)

This gives 4π Closure a second special function.

Its first function is high-risk commitment.

Its second function is selection integrity.

It protects the quality of what the system treats as a successful precedent.

20.9 The complete runtime cycle

The full architecture is:

Purpose Declaration
→ Protocol Compilation
→ Risk Diagnosis
→ Control-Subset Selection
→ Proto-Eight Actuation
→ Controlled Execution
→ Closure Audit
→ Commit or Repair
→ Trace + Residual
→ Selection
→ Admissible Revision. (20.21)

In operator form:

AgentRuntime_P = U_a ∘ Ledger_P ∘ Close_P ∘ Execute_{S_P,A_P} ∘ Compile_P. (20.22)

For high-risk tasks:

AgentRuntime_P^{high} = U_a ∘ Ledger_P ∘ Close4π,P ∘ Execute_{S_P,A_P} ∘ Compile_P. (20.23)

20.10 Final definitions

A stable agent is:

StableAgent_P = Capability_P ∧ ObjectiveAdmissibility_P ∧ AdequateSubset_P ∧ Closure_P ∧ LedgerIntegrity_P ∧ Recovery_P. (20.24)

A 4π agent is:

4πAgent_P = StableAgent_P ∧ HiddenFrameClosure_P. (20.25)

An improving agent is:

ImprovingAgent_P = 4πAgent_P ∧ AdmissibleRevision_P(L_P,R_P). (20.26)

An adaptive agent ecosystem is:

AdaptiveAgentEcosystem = Variation → Execution → 4πValidation → Selection → Retention → AdmissibleRevision. (20.27)

20.11 Final thesis

The strongest defensible conclusion is not that AI Agents are quantum systems.

It is not that nature directly supplies a finished software architecture.

It is this:

> Stable self-organizing systems repeatedly require identity preservation, bounded interaction, selective transition, binding, trace, invariance, and closure. Fundamental physics provides one mature source grammar for these roles. Proto-Eight Dynamics supplies an actuation grammar for converting potential into directed and retained

change. 成界之學 explains how projection becomes committed trace and residual. 4π Closure distinguishes visible completion from hidden-frame return. Admissible revision converts ledgered history into governed improvement.

The article can therefore close with the following summary:

> **The twelve controls provide the stability repertoire. Purpose and environment select the subset. Proto-Eight Dynamics supplies the actuation grammar. 4π Closure governs high-risk commitment and protects selection integrity. The ledger turns closure into history. Admissible self-revision turns history into improvement.**

Appendix A — Master Reference Table of the Twelve Controls

Code	Physics-derived role	AI control obligation	Typical observable	Typical failure
C1	Symmetry	Preserve meaning under equivalent surface changes	Output variance under paraphrase	Prompt brittleness
C2	Gauge/frame invariance	Preserve governed relations across representations	Cross-frame agreement	Frame contradiction
C3	Conservation	Preserve goal, permission, scope, provenance	Invariant ledger	Task drift
C4	Quantization	Separate lifecycle states	State-transition log	Half-commitment
C5	Exclusion	Prevent role overlap	Ownership map	Responsibility collision
C6	Energy gap	Require threshold before transition	Gate margin	Noise-triggered action
C7	Locality	Bound influence and access	Influence graph	Context contamination
C8	Least action	Minimize total expected execution cost	Tool calls, latency, compute	Verification churn
C9	Topology/ holonomy	Detect path-dependent residue	Route-difference audit	Hidden twist
C10	Decoherence/ commit	Convert alternatives into one stable record	Commit record	Unresolved alternatives hidden
C11	Binding/ confinement	Attach claim, evidence, artifact, and provenance	Claim–evidence graph	Citation or artifact drift
C12	Transition gate	Regulate consequential state change	Authorization record	Unsafe irreversible action

---

Appendix B — Usage-to-Control Subset Matrix

Legend:

H = high-strength control.
M = medium-strength control.
L = light control.
— = normally inactive or externally supplied.

Control	Chat	Brainstorm	Research	Coding	Legal/ Accounting	Tool Action	Multi-Agent	Adaptive
C1 Symmetry	L	L	H	M	M	L	M	M
C2 Frame invariance	—	L	H	M	H	M	H	H
C3 Conservation	M	L	H	H	H	H	H	H
C4 Quantization	L	—	H	H	H	H	H	H
C5 Exclusion	—	—	L	M	M	M	H	H
C6 Energy gap	—	—	M	H	H	H	M	H
C7 Locality	M	L	H	H	H	H	H	H
C8 Least action	H	H	M	H	M	M	H	H
C9 Holonomy	—	—	H	H	H	H	H	H
C10 Commit	L	—	H	H	H	H	H	H
C11 Binding	L	—	H	H	H	H	H	H
C12 Transition gate	L	—	H	H	H	H	H	H

This matrix is illustrative.

The actual profile must be compiled from protocol P_AI.

Appendix C — Minimal 4π Commit Checklist

A candidate may enter 4π Commit Mode only after answering the following questions.

C.1 Purpose

- Is the original task still identifiable?
- Did the scope change?
- Were changes declared?
- Is the result still within authority?

C.2 Evidence

- Are material claims attached to evidence?
- Does the evidence support the exact claim?
- Was source context preserved?
- Are conflicts disclosed?

C.3 Assumptions

- Which assumptions were necessary?
- Which are externally supplied?
- Which remain unverified?
- Would changing them alter the conclusion?

C.4 Frame robustness

- Does an equivalent rephrasing preserve the core result?
- Does a stakeholder reversal expose hidden asymmetry?
- Does another schema produce the same governed relation?

C.5 Path

- Can the major execution steps be replayed?
- Were tool calls within scope?
- Did any handoff introduce incompatible assumptions?
- Did a correct endpoint arise through an invalid route?

C.6 Residual

- What remains unresolved?
- Is it attached to the result?
- Does it require later action?
- Is it being hidden merely to create apparent completeness?

C.7 Transition

- Is the action authorized?
- Is the target verified?
- Is the consequence understood?
- Is rollback or repair available?

The final condition is:

$4\pi\text{Commit_P} \Leftrightarrow \text{PurposePass_P} \wedge \text{EvidencePass_P} \wedge \text{AssumptionPass_P} \wedge \text{FramePass_P} \wedge \text{PathPass_P} \wedge \text{ResidualPass_P} \wedge \text{TransitionPass_P}.$ (C.1)

Appendix D — Experimental Protocol

D.1 Benchmark declaration

For every task, publish:

$P_{test} = (B, \Delta, h, u, U, E, R, W)$. (D.1)

Also record:

- model;
- system prompt;
- tools;
- source set;
- random seed;
- control subset;
- control depth;
- closure mode;
- thresholds;
- human-review rule.

D.2 Experimental arms

At minimum, compare:

Arm A = Endpoint-only agent. (D.2)

Arm B = Generic self-critique agent. (D.3)

Arm C = Purpose-matched control subset. (D.4)

Arm D = Purpose-matched subset + 4π Closure. (D.5)

Arm E = Always-on twelve-control profile. (D.6)

D.3 Main outputs

Measure:

- task success;
- false commits;
- hidden-twist detection;
- residual disclosure;

- control overhead;
- recovery;
- user effort;
- inherited failure.

D.4 Main decision rule

A purpose-matched 4π architecture is supported when:

Loss_D + Cost_D < min(Loss_A + Cost_A,Loss_B + Cost_B,Loss_E + Cost_E). (D.7)

D.5 Reproducibility footer

ExperimentFooter = {TaskID,P_test,Model,Tools,Seed,S_P,Depth,ClosureMode,Thresholds,Metrics,Residual,Hash}. (D.8)

Appendix E — Undercontrol and Overcontrol Diagnostic

Symptom	Likely diagnosis	Relevant controls	Typical remedy
Fluent but unsupported answer	Under-binding	C11, C9	Claim–evidence graph
Correct result from invalid path	Hidden twist	C9, 4π	Reverse reconciliation
Endless checking	Overcontrol	C8, C6	Cost threshold
Wrong file or recipient	Locality and binding failure	C7, C11, C12	Target verification
Conflicting sub-agents	Exclusion failure	C5, C3	Role ownership map
Draft treated as final	Quantization failure	C4, C10	Explicit state machine
Action taken too early	Threshold failure	C6, C12	Transition margin
Same answer changes under reframe	Frame failure	C1, C2	Cross-frame test
Failure disappears after policy update	Self-exonerating revision	Ledger + U_adm	Revision audit
Agent never adapts	Dogmatic revision	Trace, residual, U_adm	Residual-triggered review
Agent changes constantly	Noisy revision	Conservation, gate	Trust region
High performance but unstable history	False selection	4π selection gate	Validate before retention

---

Appendix F — Compact Runtime Specification

The entire framework can be compressed into the following executable specification.

F.1 Declare

Declare_P = {Purpose,Boundary,Observation,Horizon,AllowedActions,Environment,Authority}. (F.1)

F.2 Compile

S_P = SelectControls(C_{12} | Risk,Reversibility,ExternalSupply). (F.2)

A_P = SelectActuation(Gradient,Gate,Boundary,Exchange,Trigger,Guidance,Memory,Focus). (F.3)

F.3 Execute

Y_candidate = Execute(Task | S_P,A_P). (F.4)

F.4 Close

ClosureMode_P ∈ {Open,2π,4π-Light,4π-Standard,4π-Strict}. (F.5)

For 4π:

4πClosure_P = EndpointMatch_P ∧ PurposePreserved_P ∧ EvidenceBound_P ∧ FrameRobust_P ∧ TraceReplayable_P ∧ ResidualDisclosed_P. (F.6)

F.5 Commit

Commit_P ⇔ ClosurePassed_P ∧ AuthorityValid_P ∧ TransitionGate_P. (F.7)

F.6 Ledger

$L_{k+1} = L_k \oplus$ {Declaration,Trace,Residual,GateDecision,Outcome}. (F.8)

F.7 Recover

¬ClosurePassed_P ⇒ Repair ∨ NarrowScope ∨ Escalate ∨ Abort. (F.9)

F.8 Revise

$D_{k+1} = U_adm(D_k,L_k,R_k)$. (F.10)

F.9 Learn

LearningEligibleSuccess_P = EndpointSuccess_P ∧ 4πClosure_P. (F.11)

F.10 Final runtime

ImprovingAgentRuntime_P = U_adm ∘ Ledger ∘ Close4π ∘ Execute_{S_P,A_P} ∘ Compile_P. (F.12)

Appendix G — The Projection–Closure Theorem

G.1 Why endpoint checking can fail mathematically

Let X be the full execution-state space of an agent.

A full state $x \in X$ may include:

- visible answer;
- purpose state;
- evidence state;
- permissions;
- tool history;
- assumptions;
- provenance;
- residual;
- recovery state.

Let Y be the visible output space.

The user or evaluator usually observes only:

$\pi: X \to Y$. (G.1)

where π is the output projection.

Let $T \subseteq X$ be the set of genuinely acceptable full execution states.

The visible evaluator sees only:

$\pi(T) \subseteq Y$. (G.2)

An endpoint-only evaluator accepts x whenever:

$\pi(x) \in \pi(T)$. (G.3)

The question is:

When is this endpoint rule sufficient?

G.2 Fibre of an output

For any visible output $y \in Y$, define its fibre:

$\pi^{-1}(y) = \{x \in X \mid \pi(x) = y\}$. (G.4)

The fibre contains all hidden execution states that produce the same visible result.

For example, the same answer may arise from:

- valid evidence;
- fabricated evidence;
- accidental reasoning;
- unauthorized tool use;
- a correct method;
- a corrupted method.

The visible endpoint does not distinguish these states.

G.3 Saturated target sets

A target set T is saturated under π when it contains either the whole fibre or none of it.

Formally:

T is π-saturated $\Leftrightarrow$ $T = \pi^{-1}(\pi(T))$. (G.5)

This means that whenever one hidden state producing output y is valid, every hidden state producing y is also valid.

For many trivial tasks, this may approximately hold.

For example, if the task is only:

> Produce the string “Hello.”

then the hidden route may not matter.

For consequential agent tasks, it usually does not hold.

G.4 Projection–Closure Theorem

Theorem G.1

An endpoint-only evaluator is complete for target set T if and only if T is saturated under the output projection π.

Formally:

$\text{EndpointComplete}(\pi,T) \Leftrightarrow T = \pi^{-1}(\pi(T))$. (G.6)

Proof sketch

Assume T is π-saturated.

If $\pi(x) \in \pi(T)$, then $x \in \pi^{-1}(\pi(T))$.

Since:

$\pi^{-1}(\pi(T)) = T$, (G.7)

we obtain:

$x \in T$. (G.8)

Therefore visible endpoint acceptance implies full-state validity.

Conversely, assume T is not π-saturated.

Then:

$T \neq \pi^{-1}(\pi(T))$. (G.9)

Therefore, there exists some state x' such that:

$x' \notin T$, (G.10)

but:

$\pi(x') \in \pi(T)$. (G.11)

An endpoint-only evaluator accepts x' even though x' is invalid.

Therefore the evaluator is incomplete.

■

G.5 Consequence for AI Agents

The theorem gives a precise condition for when hidden-state auditing is necessary.

If validity depends on:

- evidence;
- authorization;
- provenance;
- assumptions;
- path;
- residual;
- recovery;

then T is usually not fibre-saturated.

Therefore:

EndpointMatch_P is insufficient. (G.12)

The agent must evaluate some of the hidden execution state.

This yields:

$$\text{FullAcceptance}_P \Leftrightarrow \pi(x) \in \pi(T) \wedge x \in T. \quad (G.13)$$

A practical 4π audit attempts to estimate whether x belongs to T rather than checking only π(x).

G.6 False completion

Define the false-completion set:

$$F_{false} = \pi^{-1}(\pi(T)) \setminus T. \quad (G.14)$$

These are states that look valid under output projection but are not genuinely valid.

The false-completion risk is:

$$\Pr(\text{FalseCompletion}) = \Pr(x \in F_{false}). \quad (G.15)$$

The purpose of 4π Closure is to reduce this probability.

$$4\pi\text{Value}_P \approx \Pr(x \in F_{false}) \times \text{Damage}_P \times \text{DetectionGain}_P - \text{AuditCost}_P. \quad (G.16)$$

G.7 Why this theorem matters

This theorem replaces the vague claim:

> Hidden state may matter.

with a sharper statement:

> Hidden-state checking is mathematically necessary whenever target validity is not constant across output fibres.

This is the formal foundation of the 4π architecture.

The theorem does not prove that every agent needs strict 4π Closure.

It proves that endpoint-only evaluation is complete only under a restrictive structural condition.

Appendix H — From Literal 4π Periodicity to General Lifted Closure

H.1 Literal spinor closure

For a spin-½ representation:

$$U(\theta) = \exp(-i\theta n \cdot \sigma/2). \quad (H.1)$$

Then:

$U(2\pi) = -I$. (H.2)

And:

$U(4\pi) = +I$. (H.3)

A 2π rotation returns the visible orientation but changes the spinor sign.

A 4π rotation restores the lifted spinor state.

This structure is associated with a double cover:

$SU(2) \to SO(3)$. (H.4)

The hidden state has two sheets.

H.2 Exact AI 4π structure

An exact spinor-style AI analogy would require:

1. a visible state space Y;
2. a richer state space X;
3. a two-sheet covering map $\pi: X \to Y$;
4. a hidden class $g \in \mathbb{Z}_2$;
5. a loop that changes g after one traversal;
6. a second traversal that restores g.

Formally:

$\text{Holonomy}(\gamma) = -1$. (H.5)

$\text{Holonomy}(\gamma^2) = +1$. (H.6)

Only then would strict 4π periodicity be mathematically justified.

Most current AI workflows do not yet demonstrate this exact structure.

H.3 General lifted closure

The more general architecture is:

$\text{LiftedClosure}_P = \text{VisibleClosure}_P \wedge \text{HiddenStateClosure}_P$. (H.7)

The hidden state may be:

- binary;
- continuous;
- vector-valued;

- graph-valued;
- probabilistic;
- group-valued;
- category-valued.

Therefore:

$4\pi\text{Closure} \subset \text{LiftedClosure}$. (H.8)

The name 4π remains useful as a memorable prototype, but the general theory should be called **lifted closure** or **hidden-frame closure** when exact spinor structure is not established.

H.4 Closure taxonomy

Closure type	Visible state	Hidden state	Typical use
Endpoint closure	Output only	Ignored	Low-risk generation
State closure	Output + lifecycle state	Finite state	Workflow control
Evidence closure	Output + claim–evidence graph	Graph	Research and law
Permission closure	Output + authority path	Access state	Tool agents
Provenance closure	Output + source lineage	Directed acyclic graph	Data and code
Holonomy closure	Output + path residue	Group or groupoid element	Long execution chains
Spinor 4π closure	Output + $\mathbb{Z}_2$ sheet	Double cover	Exact special case
Recursive closure	Output + revised declaration	Self-revising ledger	Adaptive agents

---

H.5 Groupoid closure

For complex agent workflows, a groupoid may be more appropriate than a group.

Let objects represent runtime contexts:

$\text{Obj}(G) = \{\text{PurposeState}, \text{ToolState}, \text{EvidenceState}, \text{PermissionState}\}$. (H.9)

Let morphisms represent admissible transitions.

A path is:

$\gamma = f_n \circ f_{n-1} \circ \dots \circ f_1$. (H.10)

Closure requires the composite path to return to an admissibly equivalent object:

$\text{Target}(\gamma) \cong \text{Source}(\gamma)$. (H.11)

The equivalence is not necessarily exact identity.

It may preserve:

- purpose;
- authority;
- evidence;
- artifact identity;
- residual obligations.

This is likely closer to real agent execution than a single binary twist.

H.6 Terminology rule

The article should use:

- **4π Closure** for the engineering metaphor and special architecture;
- **lifted closure** for the general mathematical condition;
- **spinor closure** only where a genuine double-cover structure is explicitly modeled.

This keeps the article technically cautious without losing the central intuition.

Appendix I — Proto-Eight Actuation × Twelve-Control Stability Matrix

I.1 Why the two grammars should be mapped

Proto-Eight Dynamics and the twelve controls solve different problems.

Proto-Eight asks:

> How does the system move?

The twelve controls ask:

> How does movement remain governed?

Their interaction can be represented as a matrix.

Let:

$A_8 = \{\text{Gradient},\text{Gate},\text{Boundary},\text{Exchange},\text{Trigger},\text{Guidance},\text{Memory},\text{Focus}\}$. (I.1)

Let:

$C_{12} = \{C1,\ldots,C12\}$. (I.2)

Define interaction matrix M:

M_{ij} = relevance of control C_j to actuation role A_i. (I.3)

I.2 Mapping table

Legend:

H = strong relation.
M = moderate relation.
L = weak relation.

Proto-Eight role	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
Gradient	L	M	H	L	—	H	L	H	L	—	M	L
Gate	L	M	H	H	M	H	H	M	M	H	H	H
Boundary	M	H	H	M	H	M	H	L	M	M	H	H
Exchange	M	H	H	M	H	M	H	H	H	M	H	H
Trigger	L	L	M	H	M	H	H	M	M	H	M	H
Guidance	H	H	H	M	M	M	H	H	H	M	H	M
Memory	M	H	H	H	M	M	H	M	H	H	H	H
Focus	H	M	H	M	L	H	H	H	M	M	M	M

I.3 Gradient and conservation

A gradient creates pressure to move.

But gradients can become misaligned.

For an agent:

g_P = desired direction of improvement. (I.4)

Conservation asks whether the gradient remains attached to the declared purpose.

PurposeAlignedGradient_P ⇔ g_P · ∇J_P > 0. (I.5)

A proxy gradient may increase visible performance while decreasing real purpose integrity.

I.4 Gate and transition controls

The Proto-Eight gate corresponds strongly to:

- C4 quantization;
- C6 energy gap;
- C10 commitment;
- C12 transition gate.

A complete gate has four layers:

GateComplete = StateKnown ∧ ThresholdMet ∧ ClosurePassed ∧ AuthorityValid. (I.6)

I.5 Boundary and locality

Proto-Eight boundary and C7 locality are closely related but not identical.

Boundary defines:

What is inside? (I.7)

Locality defines:

How may influence propagate inside and across the boundary? (I.8)

Therefore:

Boundary_P = declared domain. (I.9)

Locality_P = permitted influence graph within that domain. (I.10)

I.6 Memory and trace

Proto-Eight memory stores retained influence.

The twelve-control grammar adds:

- provenance;
- binding;
- lifecycle;
- frame consistency;
- admissible transition.

Memory without control may preserve misinformation.

ControlledMemory_P = RetainedTrace_P ∧ Provenance_P ∧ Scope_P ∧ RevisionRule_P. (I.11)

The Proto-Eight engineering framework's memory–focus dyad and the declared-disclosure sequence both support this interpretation: memory is not passive storage but retained structure shaping later selection and action.

I.7 Focus and least action

Focus allocates scarce attention.

Least action controls the total cost of that allocation.

Let a_i be attention assigned to item i.

Let v_i be expected informational value.

The focus problem is:

$a^* = \arg\max_a \Sigma_i v_i a_i$ subject to $\Sigma_i a_i \leq B_attention$. (I.12)

This can reduce unnecessary verification while preserving scrutiny of load-bearing claims.

Appendix J — Control Dependency Graph and Candidate Basis Reduction

J.1 Why a flat list may be misleading

The twelve controls are often presented as a list.

But they interact as a dependency graph.

For example:

- commitment depends on lifecycle state;
- evidence closure depends on binding;
- path closure depends on trace;
- transition gating depends on authority and state;
- frame invariance depends on a stable declared object.

Define the control graph:

$$G_C = (V_C,E_C). \quad (J.1)$$

where:

$$V_C = C_{12}. \quad (J.2)$$

An edge:

$$C_i \to C_j \quad (J.3)$$

means that C_j depends partly on C_i.

J.2 Candidate dependency structure

A plausible structure is:

C3 Conservation → C4 Quantization. (J.4)

C4 Quantization → C10 Commitment. (J.5)

C10 Commitment → C12 Transition. (J.6)

C11 Binding → C9 Holonomy Audit. (J.7)

C7 Locality → C5 Exclusion. (J.8)

C1 Symmetry → C2 Frame Invariance. (J.9)

C6 Energy Gap → C12 Transition. (J.10)

C8 Least Action constrains all active controls. (J.11)

This suggests that C8 is a global economic regularizer rather than an isolated function.

J.3 Candidate architectural layers

The twelve controls may be grouped into four layers.

Representation layer

$R_C = \{C1,C2\}$. (J.12)

Integrity layer

$I_C = \{C3,C5,C7,C11\}$. (J.13)

Transition layer

$T_C = \{C4,C6,C10,C12\}$. (J.14)

Global path layer

$G_C = \{C9\}$. (J.15)

Economic regularizer

$E_C = \{C8\}$. (J.16)

This yields:

$C_{12} = R_C \cup I_C \cup T_C \cup G_C \cup E_C$. (J.17)

J.4 Candidate minimal basis

A possible smaller basis is:

$C_{basis} = \{Invariance,Conservation,Boundary,Threshold,Binding,Trace,Commit\}$. (J.18)

The twelve controls may then be derived as specializations.

For example:

$Symmetry \subset Invariance$. (J.19)

$Exclusion \subset Boundary + Identity$. (J.20)

$EnergyGap \subset Threshold$. (J.21)

$Quantization \subset State + Threshold$. (J.22)

$TransitionGate \subset Commit + Authority$. (J.23)

$Holonomy \subset Trace + Path$. (J.24)

This reduction is only a research hypothesis.

It should be tested through ablation and factor analysis.

J.5 Basis-reduction experiment

Construct a matrix:

$F \in \mathbb{R}^{m \times 12}$. (J.25)

where:

F_{ij} = effectiveness of control C_j against failure class f_i. (J.26)

Apply:

- rank analysis;
- clustering;
- principal components;
- non-negative matrix factorization;
- ablation.

If:

$\operatorname{rank}(F) < 12$, (J.27)

then the twelve controls may contain redundancy.

If all twelve contribute unique high-load-bearing coverage, the full repertoire is strengthened.

Appendix K — Declaration, Trace, Residual, and Revision Schemas

K.1 Purpose

An implementable agent needs structured records.

Natural-language logs are useful but insufficient for reliable automation.

The following schemas convert the framework into machine-readable objects.

K.2 Declaration schema

```
DeclarationRecord
{
  declaration_id
  episode_id
  purpose
  goal
  prohibited_outcomes
  boundary
  observation_rule
  horizon
```

```
  admissible_actions
  environment
  runtime
  human_workflow
  authority
  baseline_q
  feature_map_phi
  success_criteria
  control_profile
  closure_mode
  trace_rule
  residual_rule
  recovery_rule
  revision_authority
}
```

In formal notation:

$D_k = (q_k, \varphi_k, P_k, \hat{O}_k, \text{Gate}_k, \text{TraceRule}_k, \text{ResidualRule}_k)$. (K.1)

The declaration sequence is consistent with the project's declared pre-time field framework, where filtration, projection, gate, trace, and residual become meaningful only under an explicit declaration.

K.3 Control-profile schema

```
ControlProfile
{
  control_id
  active
  depth
  implementation_module
  external_supply
  activation_reason
  failure_classes
  threshold
  audit_metric
  fallback
}
```

For each C_i:

$\text{Profile}(C_i) = (\text{Active}_i, \text{Depth}_i, \text{Module}_i, \text{Supply}_i, \text{Threshold}_i, \text{Metric}_i)$. (K.2)

K.4 Claim–evidence schema

```
ClaimEvidenceRecord
{
  claim_id
  claim_text
  claim_type
  materiality
  source_ids
  support_relation
  evidence_strength
  assumptions
  counterevidence
  frame_sensitivity
  residual
  reviewer_status
```

```
}
```

Support relation:

$B(c_i,e_j) \in \{\text{supports,qualifies,contradicts,contextualizes,insufficient}\}$. (K.3)

K.5 Trace schema

```
TraceRecord
{
  trace_id
  episode_id
  timestamp
  state_before
  action
  tool
  input_reference
  output_reference
  authority
  state_after
  control_checks
  gate_result
  provenance
  hash
}
```

Trace update:

$L_{k+1} = L_k \oplus \text{TraceRecord}_k$. (K.4)

K.6 Residual schema

```
ResidualRecord
{
  residual_id
  episode_id
  category
  description
  severity
  affected_claims
  cause
  known_unknown
  unknown_unknown_proxy
  next_action
  owner
  expiry_or_review_date
  closure_status
}
```

Residual status:

$\text{Status}(R_i) \in \{\text{Open,Accepted,Deferred,Escalated,Resolved}\}$. (K.5)

A residual should never disappear merely because the episode ended.

K.7 Gate schema

```
GateDecision
{
  gate_id
  source_state
  target_state
  required_conditions
  observed_conditions
  threshold
  closure_mode
  pass
  authority
  reason
  rollback_available
}
```

Transition rule:

State_j permitted ⇔ GateDecision.pass = true. (K.6)

K.8 Revision schema

```
RevisionRecord
{
  revision_id
  previous_declaration_id
  new_declaration_id
  triggering_trace_ids
  triggering_residual_ids
  fields_changed
  justification
  expected_benefit
  risk
  frame_test
  rollback_plan
  approver
  adoption_status
}
```

The revision operator is:

$D_{k+1} = U_a(D_k, L_k, R_k)$. (K.7)

The self-revising declaration framework requires such revision to remain trace-preserving, residual-honest, frame-robust, budget-bounded, and non-degenerate.

Appendix L — Worked Runtime Profiles

L.1 Low-risk teaching assistant

Purpose

Explain a concept clearly without external action.

Profile

S_teach = {C1,C2_light,C3,C7,C8,C10_light}. (L.1)

Closure

2π or light 4π.

Ledger

- topic;
- explanation path;
- uncertainty;
- examples used.

Main residual

Possible simplification error.

L.2 Evidence-based research assistant

Purpose

Produce a publication-ready synthesis.

Profile

S_research = {C1,C2,C3,C4,C7,C9,C10,C11,C12}. (L.2)

Closure

Standard or strict 4π.

Ledger

- source list;
- claim–evidence map;
- assumptions;
- source disagreement;
- residual.

Commit rule

Publish ⇔ EvidenceBound ∧ FrameRobust ∧ TraceReplayable ∧ ResidualAttached. (L.3)

L.3 Database-writing agent

Purpose

Update a declared set of records.

Profile

S_db = {C3,C4,C5,C6,C7,C9,C10,C11,C12}. (L.4)

Environment supply

- transaction;
- schema;
- access control;
- rollback.

Strict conditions

TargetSetVerified = 1. (L.5)

RowCountExpected ≈ RowCountActual. (L.6)

RollbackReady = 1. (L.7)

AuthorityValid = 1. (L.8)

Closure

Strict 4π action closure.

L.4 Coding agent with CI/CD

Purpose

Modify code while preserving behavioral intent.

Profile

S_code = {C3,C4,C6,C7,C8,C9,C10,C11,C12}. (L.9)

Environment supply

- compiler;
- tests;
- version control;
- pipeline;
- rollback.

Commit rule

Merge ⇔ RequirementPreserved ∧ TestsPassed ∧ ScopeValid ∧ DependencyAudit ∧ RollbackAvailable. (L.10)

L.5 Multi-agent research team

Roles

- planner;
- retriever;
- analyst;
- critic;
- integrator.

Profile

S_multi = {C2,C3,C4,C5,C7,C9,C10,C11,C12}. (L.11)

Role map

Owner(Task_i) = exactly one primary agent. (L.12)

Reviewers(Task_i) ≥ one independent agent. (L.13)

Global closure

Global4π = ParentGoalPreserved ∧ HandoffConsistency ∧ EvidenceCompatibility ∧ ResidualMerged. (L.14)

L.6 Self-revising adaptive agent

Purpose

Improve future execution policy.

Profile

S_adapt = S_task ∪ {TraceLedger,ResidualLedger,RevisionGate,Rollback}. (L.15)

Revision condition

Adopt(Π′) ⇔ PerformanceGain ∧ TracePreserved ∧ ResidualHonest ∧ FrameRobust ∧ RollbackReady. (L.16)

Forbidden revision

SelfExoneratingRevision = true ⇒ Reject. (L.17)

Appendix M — Minimal Implementation Pseudocode

M.1 Control compilation

```
function compile_runtime(requirement, environment):
    declaration = declare(requirement, environment)
    risk = assess_risk(declaration)
    external_supply = inspect_environment_controls(environment)

    profile = {}
    for control in CONTROL_REPERTOIRE:
        demand = estimate_control_demand(control, declaration, risk)
        need = max(demand - external_supply[control], 0)

        if need > activation_threshold(control):
            profile[control] = choose_depth(control, need, risk)

    actuation = compile_proto_eight_actuation(declaration)
    closure_mode = choose_closure_mode(risk, declaration)
    ledger_rules = compile_ledger_rules(declaration, closure_mode)

    return RuntimeKernel(
        declaration,
        profile,
        actuation,
        closure_mode,
        ledger_rules
    )
```

M.2 Execution

```
function execute_kernel(kernel):
    state = INITIALIZED
    trace = []
    residual = []

    while state not in TERMINAL_STATES:
        action = select_action(kernel, state)
        check_local_controls(action, kernel.profile)

        result = perform(action)
        trace.append(record(action, result))

        residual.extend(detect_residual(result, kernel))

        state = transition(state, result, kernel)

    return CandidateResult(state, trace, residual)
```

M.3 4π closure

```
function four_pi_close(candidate, kernel):
    purpose_pass = check_purpose(candidate, kernel.declaration)
    evidence_pass = check_claim_evidence(candidate)
    assumption_pass = check_assumptions(candidate)
    frame_pass = run_frame_tests(candidate)
    path_pass = replay_trace(candidate.trace)
    residual_pass = check_residual_governance(candidate.residual)
    transition_pass = check_commit_authority(candidate, kernel)
```

```
    passed = all([
        purpose_pass,
        evidence_pass,
        assumption_pass,
        frame_pass,
        path_pass,
        residual_pass,
        transition_pass
    ])

    return ClosureDecision(passed, diagnostic_vector())
```

M.4 Revision governance

```
function propose_revision(declaration, trace, residual):
    candidate = derive_revision(declaration, trace, residual)

    checks = {
        "trace_preserving": preserves_trace(candidate),
        "residual_honest": preserves_residual(candidate),
        "frame_robust": test_revision_frames(candidate),
        "budget_bounded": estimate_cost(candidate) <= REVISION_BUDGET,
        "non_degenerate": not makes_everything_pass(candidate),
        "rollback_ready": has_rollback(candidate)
    }

    if all(checks.values()):
        return APPROVE(candidate)

    return REJECT(candidate, checks)
```

Appendix N — Formal Claim Ladder

N.1 Why claim strength matters

The article crosses:

- physics;
- AI engineering;
- systems theory;
- philosophy;
- Meme Thermodynamics.

Without a claim ladder, readers may confuse:

- metaphor;
- structural correspondence;
- operational equivalence;
- formal isomorphism;
- physical identity.

N.2 Claim levels

Level 0 — Metaphorical resemblance

A resembles B in an intuitive way.

$A \sim B$. (N.1)

Level 1 — Functional analogy

A and B perform similar roles under different substrates.

$\text{Role}(A) \approx \text{Role}(B)$. (N.2)

Level 2 — Structural homomorphism

There exists a relation-preserving map:

$f: A \to B$. (N.3)

such that:

$f(a_1 \circ a_2) = f(a_1) \odot f(a_2)$. (N.4)

Level 3 — Behavioral simulation

System B can reproduce the relevant behavior of A under declared observations.

$\text{Obs}_B(\text{Sim}(A)) \approx \text{Obs}_A(A)$. (N.5)

Level 4 — Bisimulation

A and B mutually simulate each other under a declared transition relation.

$A \approx_{\text{bisim}} B$. (N.6)

Level 5 — Isomorphism

There exists a bijective structure-preserving map.

$A \cong B$. (N.7)

Level 6 — Physical identity

A and B are the same physical kind of system.

$A =_{\text{physical}} B$. (N.8)

N.3 Recommended claim level for this article

The defensible main claim is Level 1 to Level 2:

Physics-derived roles provide a functional and potentially structure-preserving grammar for agent control.

The article should not claim Level 5 or Level 6 without formal evidence.

The Self-Organization Substrate Principle raises a stronger cross-scale thesis, but it also explicitly distinguishes structural recurrence from literal physical identity.

Appendix O — Terminology Guide

Term	Meaning in this article
Control repertoire	Full set of candidate stability functions
Control subset	Controls activated for one protocol
Control depth	Advisory, structured, or enforced implementation
Purpose	Declared target plus constraints and success conditions
Protocol	Boundary, observation rule, horizon, and admissible interventions
Actuation	Mechanism converting possibility into directed action
Closure	Condition under which an episode may count as complete
2π completion	Visible endpoint match
4π closure	Visible completion plus hidden-frame closure
Lifted closure	General hidden-state closure beyond literal spinor structure
Trace	Retained consequence affecting future processing
Residual	Unresolved structure remaining after closure
Ledger	Persistent trace and residual record
Admissible revision	Trace-preserving, residual-honest change to future declaration
False completion	Visible success with invalid hidden execution state
False inheritance	Retention of a strategy that succeeded only superficially
Control compiler	Process selecting controls from purpose, environment, and risk
Runtime kernel	Executable representation of declaration, controls, closure, and ledger
Proto-Eight actuation	Gradient, gate, boundary, exchange, trigger, guidance, memory, focus
World formation	Process by which candidate possibility becomes committed trace

---

Appendix P — Article-Level Publication Checklist

P.1 Conceptual integrity

- Are the twelve controls presented as a candidate repertoire rather than proven universal laws?
- Is 4π Closure distinguished from literal quantum spinor identity?
- Is purpose-matched subset selection clearly explained?
- Is P8D separated from the twelve-control stability grammar?
- Is admissible revision separated from ordinary runtime execution?

P.2 Mathematical integrity

- Are all equations single-line and Blogger-ready?
- Are symbols defined before use?
- Are protocol-relative claims marked with subscript P?
- Are analogy, homomorphism, and isomorphism kept distinct?
- Is the Projection–Closure Theorem stated with its saturation condition?

P.3 Engineering integrity

- Are external controls distinguished from internal controls?
- Are control depths specified?
- Are recovery and rollback included?
- Are false-rejection and control-cost risks acknowledged?
- Is human review treated as a bounded control rather than a guarantee?

P.4 Empirical integrity

- Are hypotheses falsifiable?
- Are baseline architectures specified?
- Are ablation studies included?
- Are control overhead and economic viability measured?
- Are residual and failure cases published?

P.5 Philosophical integrity

- Is stability distinguished from goodness?
- Is improvement distinguished from higher score?
- Is trace preservation distinguished from total surveillance?
- Is the universe–agent comparison made at the subsystem level?
- Is the strongest substrate thesis labeled as a research hypothesis?

Appendix Q — Final One-Page Architecture Summary

Q.1 Core stack

Purpose_P
→ Declaration_P
→ Protocol_P
→ ControlSubset_P

→ ProtoEightActuation_P
→ ControlledExecution_P
→ ClosureMode_P
→ Commit or Repair
→ Trace_P + Residual_P
→ Selection_P
→ AdmissibleRevision_P. (Q.1)

Q.2 Stability equation

StableAgent_P = Capability_P ∧ ObjectiveAdmissibility_P ∧ AdequateSubset_P ∧ Closure_P ∧ LedgerIntegrity_P ∧ Recovery_P. (Q.2)

Q.3 4π equation

4πCompletion_P = EndpointMatch_P ∧ PurposePreserved_P ∧ EvidenceBound_P ∧ AssumptionCoherent_P ∧ FrameRobust_P ∧ TraceReplayable_P ∧ ResidualDisclosed_P. (Q.3)

Q.4 Improvement equation

ImprovingAgent_P = StableAgent_P ∧ 4πValidatedLearning_P ∧ AdmissibleRevision_P. (Q.4)

Q.5 Control-selection equation

$\text{InternalNeed}_i(P) = [\text{Demand}_i(P) - \text{ExternalSupply}_i(P)]_+$. (Q.5)

Q.6 Selection-integrity equation

LearningEligibleSuccess_P = EndpointSuccess_P ∧ 4πClosure_P. (Q.6)

Q.7 Projection–closure condition

Endpoint evaluation is complete only when:

$T = \pi^{-1}(\pi(T))$. (Q.7)

When this condition fails:

HiddenStateAudit is necessary. (Q.8)

Q.8 Final summary

The twelve controls provide the stability repertoire.

Proto-Eight Dynamics provides the actuation grammar.

Purpose and environment select the active subset.

4π Closure distinguishes visible completion from hidden-frame return.

The ledger converts closure into historical consequence.

Admissible self-revision converts historical consequence into governed improvement.

Together:

StableImprovingAgent = ControlledActuation + LiftedClosure + LedgeredRevision. (Q.9)

The article is now **substantively complete**.

It contains:

- the revised title, subtitle, abstract, and reader contract;
- the universe–subsystem–agent comparison;
- the distinction between the twelve-control stability grammar, Proto-Eight actuation grammar, 成界之學 world-formation grammar, and 4π Closure;
- formal definitions of all twelve controls;
- relative completeness, profile adequacy, and profile minimality;
- purpose- and environment-matched control compilation;
- usage-specific subsets for brainstorming, research, coding, regulated work, tool action, multi-agent systems, and adaptive agents;
- the worked 4π-centered publication kernel;
- ledgered improvement and admissible self-revision;
- 4π Closure as a selection-integrity gate;
- failure taxonomy, falsifiable hypotheses, metrics, ablations, and economic tests;
- Appendices A–Q, including the Projection–Closure Theorem, lifted-closure taxonomy, P8D/control matrix, schemas, pseudocode, claim ladder, and publication checklist.

The remaining work is editorial rather than conceptual:

1. Normalize equation numbering and symbols across all sections.
2. Remove small repetitions created by writing in installments.
3. Add internal cross-references such as “see Appendix G.”
4. Add a references and source-framework section.
5. Perform one final terminology pass so that **4π Closure**, **lifted closure**, **hidden-frame closure**, and **spinor closure** are used consistently.
6. Format the complete manuscript as one Blogger-ready article.

Reference

A Rigorous Mathematical Grammar And Checklist That Ensure Nature-Inspired Systems Are Stable, Bounded, And Economically Viable
https://osf.io/hj8kd/files/osfstorage/6a500b7bbdb5870c2c7afb69

From Fundamental Physics to Purpose-Matched AI Agents
4π Spinor Closure, Hidden Control Stacks, and Environment-Aware Runtime Design
https://osf.io/hj8kd/files/osfstorage/6a4f89f3eef0d1166c5b9338

From Physics to AI Design: A Rosetta Stone for Runtime Architecture
https://osf.io/hj8kd/files/osfstorage/69d5023f5cdefa314c3eb654

Proto-Eight Dynamics (P8D): a small, testable model of how growth actually works 【先天八卦動力學】
https://osf.io/9rdsc/files/osfstorage/68b71c00b65e7b0e352c22f6

From Interfaces to Isomorphisms: A Protocol-Bound Theory of World Formation
How Bounded Observers Turn Fields into Operational Worlds — and Why Physics, Life, Organizations, Finance, Law, and AI Reuse the Same Grammar
https://osf.io/ae8cy/files/osfstorage/69ffbfc888878a0f3e78fda2

Philosophical Interface Engineering 1 - Turning Deep Ideas into Testable Worlds, Thought Experiments, and Civilizational Tools - A New Renaissance of Philosophy after AI
https://osf.io/ae8cy/files/osfstorage/69f777e12417f21f0f1e5206

Philosophical Interface Engineering 2 - Turning Deep Ideas into Testable Worlds, Thought Experiments, and Civilizational Tools - A New Renaissance of Philosophy after AI
https://osf.io/ae8cy/files/osfstorage/69f777e12417f21f0f1e5206

Philosophical Interface Engineering 3 - Turning Deep Ideas into Testable Worlds, Thought Experiments, and Civilizational Tools - A New Renaissance of Philosophy after AI
https://osf.io/ae8cy/files/osfstorage/69f777e12417f21f0f1e5206

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT 5.5, Google AI, Gemini 3.X, NoteBookLM, X's Grok, Claude' Sonnet 4.6 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.